How to Write a Research Proposal & Documentation

Author:

komondi@strathmore.edu

Designed for Fourth Year Scientific Research Projects in BSc. Informatics and Computer Science in School of Computing and Engineering Sciences

---

Summary

A research proposal is typically written in the third person and is organized into three main chapters: Introduction, Literature Review, and Methodology. This guide provides a detailed overview of each of these core sections, along with an analysis of the additional subsections not explicitly highlighted in this summary.

Note: Some sections of the proposal, such as the Cover Page (which includes the Research Title), Declaration and Approval, Table of Contents, and Lists of Tables, Figures, Equations, and Abbreviations, will be discussed in detail during the class seminars. **Please refer to the template provided on the classroom platform.** You are encouraged to use this template to begin your write-up - it's available for free download and reuse.

Cited References

How to write the Background of your study - Dr. Rishibha Sachdev

Lesson 1: Choosing a Research Topic

Session Outline:

1. **Choosing a Research Topic:** How to identify, refine, and narrow down a topic for your study.
2. **Writing a Research Problem:** Common mistakes to avoid when formulating your research problem.
3. **Creating and Selecting a Title:** Best practices for developing and choosing an effective research title.

Choosing a Research Topic

1. **Start with the End in Mind: -** Approach your research topic by thinking backward from the final outcome. Consider the kind of contribution you want your research to make.
2. **Select a Relevant Topic: -** Choose a topic that will be of genuine interest to the scientific community and the broader research field, especially considering the expectations at the defense stage.
3. **Important Caution: -** "*Just because it has not been done before is no justification for doing it now.*" — Peter Attiwill, Editor-in-Chief, Forest Ecology and Management.

Key Questions to Ask When Selecting a Research Topic:

1. **Relevance:** Is the topic focused on something new, current, interesting, or compelling?
2. **Impact:** Does it have the potential to provide solutions to significant challenges within an organization, sector, or society?

3. **Awareness of Existing Research:** Are you familiar with the latest findings in this field? (Make sure to review and reference top journals and leading research studies in the area.)
4. **Final Check in Topic Selection:** - Will the results of your research be interesting or useful to others in the scientific community or in a related field?

If the answer to all the guiding questions is **YES**, then you have identified a strong research topic worth pursuing. Begin exploring and gathering relevant information and literature to build a solid foundation for your study.

Common Mistakes in Research Proposal Writing:

These mistakes typically fall into three main areas:

1. **Lack of a Narrative Hook:** Failing to capture interest at the beginning by providing a strong narrative backed by evidence, such as statistics or real-world examples.
2. **Unclear Knowledge Gap:** Failing to clearly identify and demonstrate the gaps in the existing body of literature.
3. **Undefined Significance:** Failing to articulate why addressing these knowledge gaps is important and how it contributes to solving the research problem.

These mistakes often arise from three key areas:

1. **Unrealistic Expectations:** Failing to understand the level of commitment, effort, and time required to successfully complete a final year project. Set realistic expectations from the start.
2. **Poor Supervisor Relationship:** Not establishing a professional, communicative relationship with your supervisor. A strong working relationship can significantly ease problem-solving and project progress.

3. **Lack of Long-Term Focus:** Losing sight of your ultimate goal. Keep the end in mind and consistently work to stay ahead in your planning and execution.

Topic Selection

Let us now have a deep dive into topic selection and creation. What are some of the strategies for Topic Formulation (Wilkinson, 1991)?

1. **Be Brief and Concise:** Avoid unnecessary words that do not add value to your title. Eliminate phrases such as "*An Approach to...,*" "*A Study of...,*" or *"An Investigation of...*"
2. **Choose a Clear Title Structure:** Use either a **single title** or a **double title**.
    a. **Example of a single title:** Optimizing Neural Networks for Real-Time Speech Recognition
    b. **Example of a double title:** Fine-Tuning T5 Models: Enhancing Question Answering Performance in Low-Resource Languages.

In all these examples, you do notice how they're short, direct, and clearly tell you the topic without needing extra phrases like "A Study of..." or "An Investigation into...".

Other things to consider include:

1. **Keep the Title Concise:** The title should be between **12 and 18 words.** Eliminate most articles (e.g., the, a, an) and prepositions where possible. Focus on including the core subject or focus of the study—the fewer words, the better, without losing clarity.
2. **Ensure Research Feasibility:** Choose a topic that can realistically be researched (given available resources, time, and access to data) and should be researched (it addresses a meaningful gap or problem in the field).
3. **Identify Your Interests**: - Focus on areas that genuinely interest or puzzle you within your field of study.

4. **Explore Trending Topics:** - In Computer Science, consider emerging and high-impact areas such as Machine Learning, Edge Computing, Quantum Computing, Internet of Things (IoT), Blockchain, and others. Choose a topic that will sustain your interest throughout the project.
5. **Identify Relevant Keywords:** - Keywords help in refining your focus. For instance, in Machine Learning, common keywords could include K-Nearest Neighbors (KNN), Classification, Regression, Recognition, among others.
6. **Define the Topic Clearly: -** Analyze the selected keywords carefully and narrow down the scope to a specific area of focus. Ensure the topic is neither too broad nor too narrow.
7. **Formulate the Topic:** - Review existing articles, research papers, and journals relevant to your area of interest. This background research will help you sharpen and clarify your topic. Take notes, paraphrase important points, and summarize key ideas from these materials—this will greatly assist in writing your literature review.

More examples of single (standalone) topics and double topics you may explore further are as follows.

Single Titles (Simple, Standalone Titles)

1. Optimizing Edge Computing Architectures for Real-Time Data Processing
2. Blockchain-Based Identity Management Systems for Secure Online Transactions
3. Improving Image Recognition Accuracy Using Deep Convolutional Neural Networks

Double Titles (Main Title: Subtitle)

1. Enhancing Multilingual Chatbots: Improving Low-Resource Language Support Using Transfer Learning

2. Cybersecurity in Smart Cities: Analyzing Threats and Developing Proactive Defense Strategies
3. Energy-Efficient IoT Networks: Designing Sustainable Solutions for Remote Monitoring

In some scenarios, you may have topics where a case study is applied and when formulating case study topics, it is good practice to follow a few important guidelines.

1. First, clearly specify the technology or solution you are applying in your study. This helps define the technical focus of your work.
2. Second, explicitly mention the case you are examining—whether it is a specific location, organization, group, or community—so that the context of your research is well understood.
3. Finally, highlight the particular impact or challenge your study aims to address.

Doing so ensures that your research is positioned as relevant and meaningful to both academic and practical audiences. See the examples below:

1. Optimizing Federated Learning Models for Healthcare Data Privacy: A Case Study of Rural Clinics in Kenya
2. Implementing Smart Contract-Based Land Ownership Verification: A Case Study of Nairobi County's Blockchain Registry System
3. Deploying Reinforcement Learning Algorithms for Personalized Learning Paths: A Case Study of Strathmore University's E-Learning Platform

To-Do: End of Lesson 1

1. Schedule a meeting with your supervisor to discuss the research area you are interested in pursuing.
2. During the meeting, agree on a research topic that aligns with the topic formulation guidelines discussed in class.

3. Once finalized, update your selected topic on the shared Google Sheet by **Thursday, 14th May 2025**.

Lesson 2: Proposal Writing

Acknowledgements

The acknowledgements section of a thesis or research proposal is where you formally express gratitude to the individuals, institutions, and organizations that significantly supported your work. Unlike a dedication, acknowledgements are formal and focused specifically on contributions related to the research.

It is not necessary to mention everyone who assisted you—only those who played an important role in your research process. For example, it would not be appropriate to thank a partner simply for providing personal support, such as preparing meals, unless their contribution is directly related to the project.

Abstract

An abstract is a concise summary of a longer piece of writing. It highlights the key points, clearly describes the content and scope, and presents an abbreviated overview of the work. Typically, abstracts are between **100 and 250 words**, with most being around **200 words**. They often follow specific patterns, which may be determined by the guidelines of the examining academic institution.

The abstract is critical because it often forms the reader's first impression of the work—and in some cases, the only impression. For example, in funding applications, some reviewers, such as members of a foundation's board of directors, may read only the abstract before making funding decisions. As a result, the abstract must not only

summarize the proposal effectively but also make a strong and persuasive impact. It is, therefore, one of the most important elements of a research proposal or thesis.

Significance of the Abstract

1. Enables readers to determine whether the full article is relevant to their interests or research.
2. Assists readers and researchers in recalling essential findings on a specific topic.
3. Provides an overview of main points, enhancing comprehension before engaging with the complete text.
4. Facilitates indexing for efficient retrieval and cross-referencing of articles.

Core Components of an Abstract

1. **Background:** A brief opening that places the research within its broader context.
2. **Objectives:** A concise statement outlining the purpose or goals of the study.
3. **Methods:** A short description of the procedures or approach taken in the project.
4. **Findings:** A summary of key results or anticipated outcomes, often refined at the final stages—such as reporting model accuracy or observed improvements.
5. **Conclusion:** A final statement highlighting the significance or impact of the results—what they reveal, solve, or contribute.

Essential Questions an Abstract Should Address

1. **Purpose**: What motivated this study, research, or project? (Begin with the end goal in mind—such as contributing to graduation or solving a specific problem.)
2. **Approach**: What was done, and by what means? (Or, what will be done and how?)

3. **Results**: What were the key findings? (Or, what outcomes are anticipated?)
4. **Implications**: What is the significance of these findings? What do they reveal or contribute?

Tips for Writing an Effective Abstract

1. Revisit your article or proposal with the goal of identifying its core components.
    - Focus on extracting key elements: purpose, methods, scope, results, conclusions, and recommendations.
    - Use the headings or table of contents to guide the abstract structure.
2. Draft your abstract from memory after reviewing the full document—avoid copying directly.
3. Rephrase ideas in your own words to ensure clarity and originality.
4. Revise your draft to:
    - Strengthen organization and flow
    - Improve transitions
    - Remove unnecessary details
    - Ensure completeness and accuracy
    - Eliminate wordiness
    - Correct grammar, spelling, and punctuation
    - Maintain consistency in tone and voice with the main document

Abstract Example:

Review the abstract below to understand how to craft one effectively. Pay close attention to the title—it is a single, well-structured line that captures the core problem, the proposed solution, and the intended implementation methods.

Improving Question Answering in Maternal Healthcare through Generative AI and Natural Language Processing

In recent scientific research, Question Answering (QA) systems have gained significant attention across various domains, including healthcare. Within maternal healthcare, QA systems offer the potential to deliver timely and accurate responses to queries concerning pregnancy, childbirth, and postpartum care. However, current QA models in this area face limitations, such as scarce training data and difficulty processing complex questions with multiple components.

This study proposes leveraging generative AI models to enhance QA performance in maternal healthcare. The approach integrates pre-trained language models with domain-specific datasets to generate contextually appropriate and informative answers. A fine-tuning pipeline is also introduced, designed to adapt these models using a small, annotated corpus of maternal healthcare questions.

Experimental results demonstrate that the proposed method outperforms existing QA models across key evaluation metrics, including accuracy, precision, and recall. The model's ability to address complex, multi-part questions positions it as a practical tool for both healthcare providers and patients. This research highlights the potential of generative AI in advancing QA capabilities, ultimately contributing to improved access to information, better patient outcomes, and more efficient healthcare delivery.

Keywords: Question Answering, Maternal Healthcare, Generative AI, Pre-trained Language Models, Fine-tuning, Complex Queries, Healthcare Innovation

Note: The final paragraph in the abstract reflects a completed project, where all experiments have been conducted and results analyzed. At the proposal stage, however, your abstract will naturally differ, as the experiments are yet to be carried out. Keep in mind that your research direction may evolve over time. Additionally, ensure that your ***keywords***—which appear at the end of the abstract—are drawn directly from the content of your abstract and are formatted in ***bold and italics*** as shown.

To-Do: End of Lesson 2:

- **Meet with your assigned supervisor** to discuss your initial project ideas. Use insights from class sessions to formulate a well-thought-out research title.
- **Develop a draft abstract** based on your proposed title and review it with your supervisor. Refer to the example provided above for guidance.
- **Document your work** using the Word template shared in class. Ensure all sections are properly filled in as instructed.

End of Session

Please take this time for **questions**, **summaries**, and **clarifications** of any **concepts** or **assigned tasks**. Ensure you leave with a clear understanding of what is expected before the next class.

Lesson 3: Chapter 1: Introduction

The introduction of a research paper or proposal provides background information and sets the context for the study. It shows how the research relates to existing literature and identifies gaps to be explored. In proposals, especially those seeking funding, the introduction must clearly communicate the research goals and significance. A strong introduction includes the research **hypothesis**[1], which guides the research design and can be formulated by asking critical questions about the topic.

1. What is the central research problem?
2. What is the topic of study related to that problem?
3. What methods should be used to analyze the research problem?

[1] A **hypothesis** is a tentative explanation based on limited evidence, used as a starting point for further investigation.

4. Why is this research important, what is its significance, and how will its outcomes affect the funders and the research society as a whole?

Chapter 1 – Research Proposal Sections

1.1 Background Information

The background section provides the context for the research, explaining its importance and setting the stage for the study. It justifies the need for the research and summarizes its objectives. Typically, this is the opening section of a proposal, article, or thesis.

How Should the Background Information Be Structured?

1. **Historical Context & Literature Overview**
    - Briefly outline key developments in the field that led to the current research topic.
    - If the study is interdisciplinary, clarify how different disciplines intersect and which aspects will be examined. (For technology-focused research, specify relevant subfields, e.g., AI, cybersecurity, or renewable energy technologies.)
2. **Identification of Gaps**
    - Summarize existing knowledge on the broad topic.
    - Clearly state unresolved issues or missing links in the literature that the study aims to address.
3. **Significance of Addressing Gaps**

- Explain why filling these gaps matters (e.g., theoretical, practical, or societal implications).

4. **Rationale & Hypothesis**
    - Provide the logical foundation for the study.
    - State the hypothesis or research questions guiding the investigation.

How to Make the Background Engaging

To maintain reader interest (including funders), structure the background as a **compelling narrative** around the research's central theme. Avoid excessive detail or digressions into broad literature reviews. Instead:

- **Build a logical flow**: Each point should connect seamlessly, guiding readers to recognize the gaps your study addresses.

Class Discussion:

Assuming your research is focusing on Sentiment Analysis on Monolingual, Multilingual and Code - Switching Twitter Corpora, how would you structure your background?

- **Context**: Start with the rise of social media (e.g., Twitter/X) as a key platform for public sentiment.
- **Focus**: Narrow down to challenges in analyzing monolingual, multilingual, and code-switched texts.
- **Gaps**: Highlight limitations in current tools (e.g., bias in monolingual models, scarcity of code-switching datasets).

- **Climax**: Emphasize why addressing these gaps matters (e.g., improving cross-cultural communication, AI fairness, credibility in fact checking).

Common Mistakes to Avoid

1. **Length Issues**
    - **Too long**: Overwhelms with unnecessary details.
    - **Too short**: Omits critical context.
    - **Solution**: Be **concise but comprehensive**—cover key developments and gaps without tangents.
2. **Ambiguity**
    - Avoid vague language; assume readers lack deep familiarity with your topic.
3. **Irrelevant Content**
    - Stay focused on the **core problem** (e.g., for sentiment analysis, avoid digressing into general NLP history).
4. **Disorganization**
    - Present ideas **chronologically** or thematically to show field progression clearly.
    - **Example:** Trace sentiment analysis evolution → current multilingual/code-switching challenges → your study's role.

Key Takeaway: A well-structured background tells a **clear, gap-driven story** that aligns with the research's novelty and significance.

Distinguishing Background Information from Literature Review

Many researchers confuse these two sections, but they serve distinct purposes:

1. **Background Information**

- **Purpose:** Establishes the *context* and *significance* of the research.
- **Content:**
  - Broad introduction to the research area.
  - Key developments leading to the current problem.
  - Clear identification of gaps and rationale for the study.
- **Structure:**
  - Thematic + chronological organization to show field evolution.
  - Focused discussion (avoid excessive detail).
- **Referencing: Heavily cited but *concise*—only essential sources to frame the problem.**

2. **Literature Review**
   - **Purpose:** Provides *evidence* supporting the study's hypothesis/objectives.
   - **Placement:** Follows the background (typically after research questions).
   - **Content:**
     - *Comprehensive* analysis of past studies (both broad and specific).
     - Critical evaluation of methodologies, findings, and trends.
     - Explicit links to how prior work justifies the current study.
   - **Depth:** More detailed than the background—expands on all referenced studies.

Practical Tips

- **Background**: Think of it as the *"trailer"* for your research—hook the reader with clarity and urgency.
- **Literature Review**: This is the *"full film"*—prove your study's necessity through rigorous evidence.

Example for Clarity:

- **Background**: Sentiment analysis has advanced in monolingual contexts Smith, (2020), but multilingual and code-switched data remain understudied Lee, (2022), limiting global applicability.
- **Lit. Review**: Smith (2020) used LSTM models for English tweets (accuracy: 85%), while Lee (2022) identified a 20% drop in accuracy for code-switched texts, attributing it to...

Note: Always cite rigorously in both sections, but tailor references to their purpose - background = foundational; lit. review = exhaustive.

1.2 Problem Statement

A **problem statement** is a concise claim that defines the specific issue your research aims to solve. It answers the question: ***What is the problem, and why does it matter?***

Purpose of a Problem Statement

1. **Narrow the Focus**: Transforms a broad concern into a clear, researchable question.
2. **Justify the Study**: Explains why the problem warrants investigation.
3. **Guide the Proposal**: Serves as the foundation for your research objectives and methodology.

A well-crafted problem statement is **brief but impactful** - typically **Half a page or 250 words** for early-stage research, though advanced work may require more detail.

Structure of a Persuasive Problem Statement

1. **The Ideal** – Describe the optimal scenario **(*how things should be*).**

   ***Example*:** *Social media platforms should accurately detect sentiment across languages to foster inclusive communication.*

2. **The Reality** – Explain the current shortfall **(*what's wrong now*)**.

   ***Example*:** *However, most sentiment analysis tools fail with multilingual and code-switched text, misclassifying 30% of non-English tweets (Lee, 2023).*

3. **The Consequences** – Propose your research's role in bridging the gap.

   ***Example*:** *This study will develop a hybrid model to improve accuracy in code-switched data, enabling equitable AI applications.*

Note:

- **Cite evidence** (e.g., "Prior studies show... [Smith 2021]") to ground your problem in existing research.
- Avoid vagueness—quantify the problem where possible (e.g., "70% of rural clinics lack access to...").

1.3 Research Objectives

Research objectives define the **specific goals** your study aims to achieve. They provide clear direction and help measure the success of your work. Objectives should be **action-oriented, measurable, and aligned** with your problem statement.

1.3.1 General Objective

The **general objective** is the **broad, overarching goal** of your research. It summarizes the primary purpose of your study in **one concise statement.**

Examples:

1. *"To develop an end-to-end speech recognition model for Fon and Igbo languages."*
2. *"To design an improved deep learning model for question-answering tasks in maternal healthcare."*
3. *"To develop a secure e-voting application using blockchain technology."*

1.3.2 Specific Objectives

These are **detailed, actionable steps** that break down the general objective into smaller, achievable tasks. They should follow the **SMART criteria** (Specific, Measurable, Achievable, Relevant, Time-bound).

Example - in case you have a topic below:

Improving QA in Maternal Healthcare with Generative AI

General Objective:

To develop an improved question-answering (QA) model for maternal healthcare using generative AI.

Specific Objectives: - This section should be in roman numbers (small).

1. To analyze the current state of Question Answering in Maternal Healthcare Domain (you should also add that the challenges in this section will be reviewed)
    a. **Explanation:** identifying limitations in accuracy and domain coverage. (Literature review + dataset audit)
    b. This will make your first subsection in the literature review.
2. To study and analyze how currently existing works have improved Question Answering in Maternal Healthcare domain
    a. **Explanation:** This will benchmark existing generative AI models (e.g., BERT, GPT) on maternal health QA tasks, measuring F1-score and response relevance. (Comparative analysis)
    b. This will be your related works in the literature review
3. To review the challenges related to currently existing Question Answering Models in Maternal Healthcare.
    a. **Explanation:** This will be the gaps in existing literature.
4. To design and develop an improved Question Answering Model fine-tuned to Maternal Healthcare Questions.
    a. **Explanation:** This will be your Conceptual Framework
5. To test and validate the proposed Question Answering model
    a. **Explanation:** This will be your system from chapter 3 to the end.

1.4 Research Questions

A research question is the specific focus of your study, derived from your research problem and shaped by your study design. By refining your research problem into a clear, actionable idea, you arrive at your research question. Typically, these questions align with and emerge from your research objectives.

Examples: Research questions can be derived from objectives as follows:

Objective: To analyze the current state of QA in the maternal healthcare domain.

Research Question: What is the current state of QA in maternal healthcare?

Objective: To study how existing works have improved QA in maternal healthcare.

Research Question: How have previous QA models advanced question answering in maternal healthcare?

1.5 Justification

Research needs a strong "**why**" – and that justification comes from understanding what's already known and what's still missing. Here's how this study fits in - (*these are just dummy examples*):

1. **Current Knowledge: - this is what we currently know**

   For Example:

   - Studies like Smith et al. (2020) developed effective QA systems for general healthcare...
   - Johnson (2021) created a model that answers cardiology questions with 85% accuracy...

2. **Identified Gaps: - this is what is missing**

   For Example:

   - No systems specialize in maternal health scenarios:
     - Can't handle questions about high-risk pregnancies (Lee, 2022)
     - Struggles with regional language variations (Garcia, 2023)

- Existing models are 20% less accurate for postpartum care questions (Chen, 2021)

3. **The Problem This Causes: - this is why this matters**

   For Example:

   - Expectant women in developing regions often get incomplete/inaccurate information...
   - Healthcare workers waste time searching through unreliable sources...

4. **Our Solution: - this is why this study helps**

   For Example:

   - Building the first QA system specifically for maternal health
   - Incorporating:
     - Regional language support
     - Verified medical guidelines
     - Case-based reasoning
   - Expected to improve answer accuracy by 30% for maternal health queries

Summary: This work will help [target users] by [expected benefit], while advancing research in [academic field].

1.6 Scope

The scope details how in-depth your study is to explore the research question and the parameters in which it will operate in relation to the study and timeframe.

Example:

This study will focus on [specific area] while operating within defined boundaries to ensure feasible completion. The research will cover:

1. **Technical Scope:**
    - Development of a [specific system/tool] using [technology/method]
    - **Example:** *"A maternal health QA system using BERT-based models, focusing on English-language queries about pregnancy complications"*
2. **Data Scope:**
    - Analysis of [data types/sources] from [time period/location]
    - **Example:** "Publicly available maternal health guidelines from WHO (2015-2023) and anonymized patient forums"
3. **Functional Scope:**
    - Implementation of [key features] but excluding [limitations]
    - **Example:** *"Will answer diagnostic questions but not provide treatment recommendations"*
4. **Time Scope:**
    - Conducted over [duration] with milestones:
        - Phase 1: Literature review (Month 1-2)
        - Phase 2: Model development (Month 3-5)

Exclusions:

- This study will not address [related but out-of-scope areas]
- Example: *"Non-English languages or pediatric care questions"*

1.7 Limitations and Delimitations

Delimitations define the boundaries you intentionally set for your study. These are choices you make to narrow your research focus, ensuring it remains manageable and relevant. **For example:**

- Studying only female patients aged 20–35 in urban hospitals
- Analyzing data from 2015–2026 to exclude outdated trends

- Using English-language sources only

Delimitations clarify what your study *will not address*—not because of constraints, but because those aspects fall outside your research goals. They help readers understand the specific lens through which you're examining the problem.

Limitations, on the other hand, are potential weaknesses in your study that arise from factors beyond your control. These might affect the validity or generalizability of your findings. Common examples include:

- A small sample size due to limited participant availability
- Potential bias in self-reported survey responses
- Missing data in historical records that couldn't be recovered

Unlike delimitations, limitations aren't choices—they're challenges you acknowledge upfront. Addressing them openly strengthens your credibility by showing awareness of your study's constraints.

Key Differences Summarized

- **Delimitations** = Intentional exclusions (*"We focus on X and exclude Y because..."*).
- **Limitations** = Unavoidable shortcomings (*"We recognize that Z may affect results because..."*).

Why Both Matter

1. **Delimitations** keep your study targeted and feasible.
2. **Limitations** contextualize your findings and guide future research.

By transparently outlining both, you help readers interpret your work accurately and trust your conclusions.

Real-World Analogy

Think of delimitations as drawing a map ("We'll explore *this* neighborhood only") and limitations as weather conditions ("Rain limited visibility during our survey"). One is planned; the other is unpredictable.

To-Do: End of Lesson 3: - complete by next class

- Use the provided template (Download from Google Classroom) to draft all sections of the proposal till end of Chapter 1:
- Keep it focused:
    - Align all sections with your research problem
    - Be specific in scope/delimitations (avoid vague wording)
- Review before submitting: - Work with your supervisor
    - Check formatting matches the template

Lesson 4: Chapter 2: Literature Review

Definition and Purpose

A literature review surveys and critically evaluates scholarly sources (books, articles, papers and journals etc.) relevant to a research problem. It summarizes, synthesizes, and contextualizes existing work, demonstrating how your research fits within the broader field.

Key Objectives:

- Situate each source's contribution to understanding the research problem.
- Analyze relationships, conflicts, and gaps among studies.
- Offer new interpretations of prior research.
- Prevent duplication by identifying existing scholarships.
- Justify the need for further research.

Structure of a Literature Review

1. **Introduction**
    - Overview of the topic, issue, or theory.
    - Objectives of the review.
2. **Thematic Organization**
    - Group sources by themes (e.g., supporting/opposing/alternative viewpoints).
    - Compare and contrast works, noting similarities and differences.
3. **Conclusion**
    - Highlight the most compelling arguments and contributions.
    - Identify unresolved gaps or contradictions.

Critical Evaluation of Sources

Assess each work by examining:

- **Provenance:** Author's credentials and evidence quality.
- **Methodology:** Appropriateness of data collection, analysis, and reporting.
- **Objectivity:** Balance of perspective and inclusion of contrary evidence.
- **Persuasiveness:** Strength of the author's arguments.
- **Value:** Contribution to the field.

Writing Tips

- **Use Evidence:** Support analysis with citations.
- **Be Selective:** Focus on key points tied to the research problem.
- **Summarize & Synthesize:** Integrate sources thematically.
- **Maintain Your Voice:** Prioritize your analytical perspective.
- **Paraphrase Carefully:** Accurately represent authors' ideas and cite properly.

Common Mistakes to Avoid

- Including irrelevant sources.
- Failing to critically assess research design or biases.
- Overlooking contrary findings or alternative interpretations.
- Relying too heavily on one type of source (e.g., only supportive studies).

Note: Write in the third person.

Class Discussions: Structuring Chapter 2

Tailoring the Literature Review to Your Objectives

Chapter 2's structure should align with your research objectives. Below is an example using the topic:

"Improving Question Answering Tasks in Maternal Healthcare Using Generative AI Models in Natural Language Processing."

Key Principle:

- Group related objectives to avoid redundancy. For instance, Objectives 1 and 2 might merge into a single subsection (e.g., *2.2: Generative AI in Healthcare*).

2.1 Introduction

This section outlines the organization of **Chapter 2 (Literature Review)**, detailing its scope, thematic structure, and the rationale behind the selected sources. It provides a roadmap for readers by:

1. **Contextualizing the Review**
    - Briefly restating the research problem (*e.g., improving QA tasks in maternal healthcare using generative AI*).
    - Explaining the chapter's role in grounding the study within existing scholarship.
2. **Describing the Layout**
    - Thematic grouping of literature (*e.g., AI in healthcare, NLP techniques, ethical considerations*).
    - Justification for merged/combined objectives (*if applicable*).

3. **Highlighting Key Sources**
    - Types of works reviewed (*peer-reviewed articles, datasets, case studies*).
    - Criteria for inclusion (*relevance, publication date, methodological rigor*).
    - Focus of papers/journals that are 3 years old (best case) or 5 years old (worst case)
4. **Transitioning to Subsections**
    - Preview of how each subsection (*2.2, 2.3, etc.*) addresses specific research objectives.

Example Phrasing:

Section 2.2 examines generative AI's applications in maternal healthcare, while Section 2.3 evaluates NLP techniques for QA tasks, culminating in identified gaps (Section 2.5).

2.2 Current State of Question Answering in Maternal Healthcare

Alignment with Objective I: *To analyze the current state of QA in maternal healthcare, identifying limitations in accuracy and domain coverage.*

2.2.1 Overview of QA in Maternal Healthcare

- **Definition**: QA systems in maternal healthcare leverage NLP to provide accurate, timely responses to medical queries (e.g., symptom checks, treatment guidelines) (Smith et al., 2022).
- **Current Approaches**:

    - Rule-based systems (e.g., WHO chatbots) with limited adaptability (Doe, 2021).
    - Generative AI (e.g., GPT-3 fine-tuned for obstetrics) improving dynamic responses (Lee et al., 2023).
- **Efficacy**:
    - **Successes**: Improved access in low-resource settings (e.g., *MamaToto* app in Uganda reduced maternal mortality by 20% (UN, 2022)).
    - **Limitations**: Poor handling of regional dialects and rare complications (Zhang et al., 2023).

2.2.2 Key Challenges ....

(Prose format with citations)

Existing QA systems face:

1. **Accuracy Gaps**: Misdiagnosis risks due to limited domain-specific training data (e.g., misclassifying preeclampsia symptoms (HealthTech, 2023)).
2. **Domain Coverage**: Over-reliance on Western datasets neglects local healthcare contexts (African Journal of Medicine, 2024).
3. **Real-World Integration**: Lack of clinician-AI collaboration frameworks (WHO, 2023).

2.3 Related Works in Maternal Healthcare QA

Alignment with Objective II: *Benchmark existing generative AI models on maternal health QA tasks.*

2.3.1 Jibu Sasa: Web-Based QA Application

- **Description**: A Kenyan platform providing maternal health advice via NLP (Android/iOS).
- **Technology**: BERT-base fine-tuned on Swahili/English medical texts (Acc.: 78% F1-score (KEMRI, 2023)).
- **Gaps**:
    - Limited multilingual support beyond East Africa.
    - No peer-reviewed validation for high-risk pregnancy queries (Mwangi et al., 2023).

(Optional: 1 screenshot of the interface in Appendix A.)

2.3.2 MediBot: Multilingual AI Assistant for Maternal Health

(Dummy Example 2 – Mobile App)

- **Description**: A cross-platform (Android/iOS) chatbot developed in Nigeria, integrating GPT-4 and WHO guidelines for prenatal care.
- **Technology**:
    - **Model**: GPT-4 fine-tuned on Nigerian Pidgin, Yoruba, and English.
    - **Performance**: 82% F1-score on local symptom queries (Nigerian Medical Journal, 2023).
- **Gaps**:

    - Struggles with rare conditions (e.g., placenta accreta).
    - No integration with hospital EHR systems.

2.3.3 MamaCare: Voice-Based QA for Rural Clinics

(Dummy Example 3 – Low-Resource Focus)

- **Description**: A voice-enabled AI tool used in Ethiopian clinics, designed for illiterate users (Amharic/Oromo).
- **Technology**:
    - **Model**: Hybrid of BERT and speech-to-text.
    - **Performance**: 75% accuracy on voice queries (Addis Ababa University, 2023).
- **Gaps**:
    - High latency (>5s response time).
    - Fails to distinguish between urgent/non-urgent queries.

Screenshot: Interface flowchart (Appendix B).

2.3.4 SafeBirthAI: High-Risk Pregnancy Advisor

(Dummy Example 4 – Specialized Use Case)

- **Description**: A web-based tool for clinicians, predicting complications (e.g., eclampsia) using RAG + Llama 2.
- **Technology**:
    - **Training Data**: 10,000+ case studies from Johns Hopkins.
    - **Benchmark**: 88% precision in risk alerts (JAMA, 2024).

- **Gaps**:
    - Requires high-bandwidth internet.
    - Limited to English-language inputs.

Table 2.2: Comparative Analysis of Reviewed Systems

System	Strengths	Critical Gaps
Jibu Sasa	Local language support	Narrow scope
MediBot	Multilingual	No EHR linkage
MamaCare	Voice accessibility	Slow response time
SafeBirthAI	High-risk focus	Infrastructure-dependent

2.4 Gaps in Related Works

Should be written in prose and cited with the relevant sections in the related works. In case you have to refer to some tables, you should cross - reference them

A critical synthesis of the reviewed systems (*Jibu Sasa, MediBot, MamaCare, SafeBirthAI*) reveals persistent limitations in maternal healthcare QA technologies, which this study aims to address:

1. **Limited Clinical Validation**

   Most tools (e.g., *Jibu Sasa*, *MediBot*) rely on narrow benchmark datasets rather than real-world clinical trials, risking inaccurate responses for high-risk scenarios like postpartum hemorrhage (Nigerian Medical Journal, 2023; KEMRI, 2023). *SafeBirthAI*'s focus on predictive alerts lacks peer-reviewed validation across diverse populations (JAMA, 2024).

2. **Language and Regional Bias**

   While *MediBot* and *MamaCare* support local languages, their training data overlooks regional dialects (e.g., Hausa variations in Northern Nigeria) and cultural contexts (e.g., traditional birth practices), reducing efficacy for rural populations (Addis Ababa University, 2023).

3. **Integration Challenges**

   None of the systems interface with Electronic Health Records (EHRs) or clinician workflows, limiting their utility in hospital settings (*Table 2.2*). *MamaCare*'s voice-based system further suffers from latency (>5s), rendering it impractical for emergency use.

4. **Infrastructure Dependencies**

   High-performance tools like *SafeBirthAI* require stable internet, excluding low-resource clinics (WHO, 2023). Conversely, offline-capable systems (*Jibu Sasa*) lack dynamic updating mechanisms, perpetuating outdated medical advice.

5. **Ethical Oversights**

   Only *SafeBirthAI* included bias audits for sensitive queries (e.g., abortion complications). Others (*MediBot*, *MamaCare*) fail to address disparities in maternal care access for marginalized groups (AI Ethics, 2023).

Key Unmet Need: A unified QA model combining *language adaptability* (like *MediBot*), *low-resource optimization* (like *MamaCare*), and *clinical rigor* (like *SafeBirthAI*) remains absent—a gap this research directly targets.

2.5 Conceptual Framework

Definition and Purpose

The conceptual framework for this study visually and theoretically outlines the development of an **improved QA model for maternal healthcare using generative AI**. It integrates key variables, relationships, and processes derived from the literature review (Sections 2.2–2.4) and aligns with the research objectives.

Steps to Develop the Framework

1. **Research Focus**
   - **Topic**: *Improving QA in Maternal Healthcare with Generative AI*.
   - **Research Question**: *How can generative AI models be optimized to address accuracy, language diversity, and clinical integration gaps in maternal healthcare QA systems?*
2. **Key Variables and Concepts**
   - **Input Variables**:

        - *User Queries*: Maternal health questions (e.g., symptom checks, prenatal advice).
        - *Data Sources*: Medical literature (WHO guidelines), EHRs, local language datasets.
    - **Processing Variables**:
        - *AI Model*: Fine-tuned generative LLM (e.g., Llama 2 with RAG).
        - *Validation*: Clinician feedback loops, bias audits.
    - **Output Variables**:
        - *Responses*: Accurate, culturally adapted QA outputs.
        - *Evaluation Metrics*: F1-score, latency, user satisfaction.
3. **Theoretical Foundation**
    - Builds on:
        - *Human-AI Collaboration Theory* (Klein et al., 2022): Ensures clinician oversight.
        - *Low-Resource NLP Models* (Ruder et al., 2023): Addresses language/dataset gaps.
4. **Visual Representation**
    - There are various ways to visually represent a conceptual framework, depending on your preference and the complexity of your research. Some common formats include flowcharts, **diagrams**, or schematic representations. You can use software like Microsoft Word, PowerPoint, or **specialized diagramming tools** to create your visual representation.
5. **Relationships Between Variables**
    - Fine-tuning the AI model on local datasets (*Input*) improves response accuracy (*Output*).

- Clinician feedback (*Processing*) reduces bias in high-risk scenarios (*Output*).

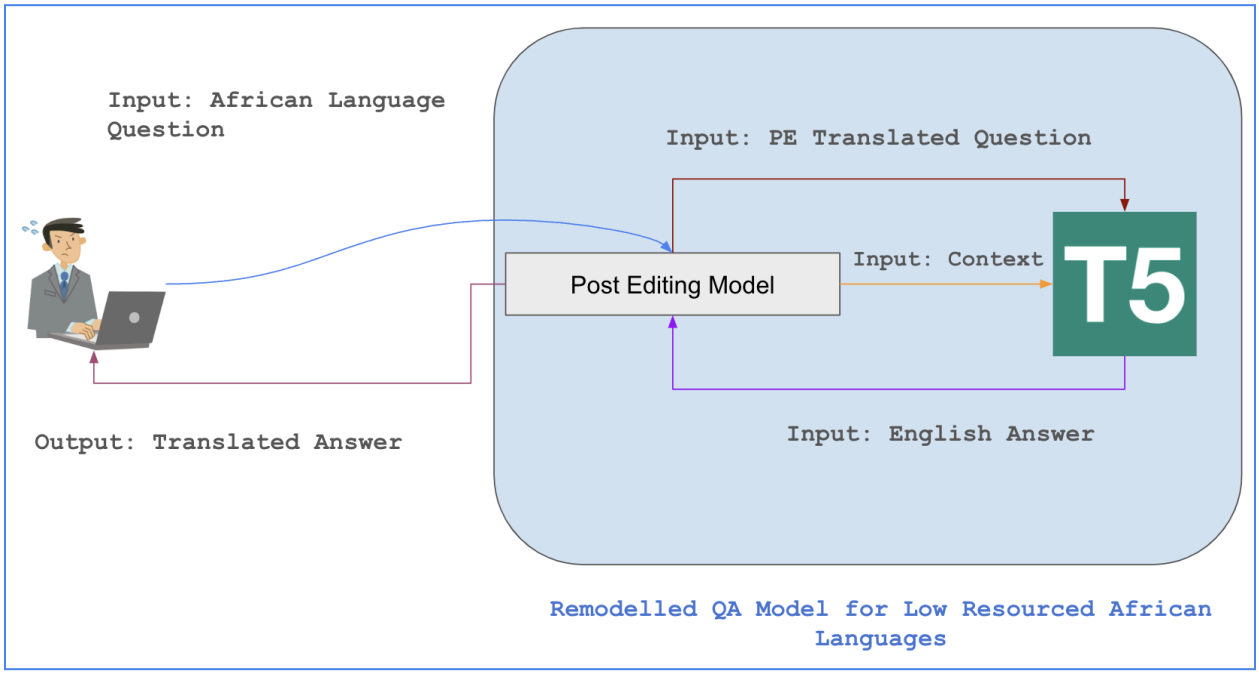
6. **Flexibility and Refinement**
    - The framework will evolve during model testing (Chapter 5) based on real-world data.

Example: Applied to Maternal Healthcare QA

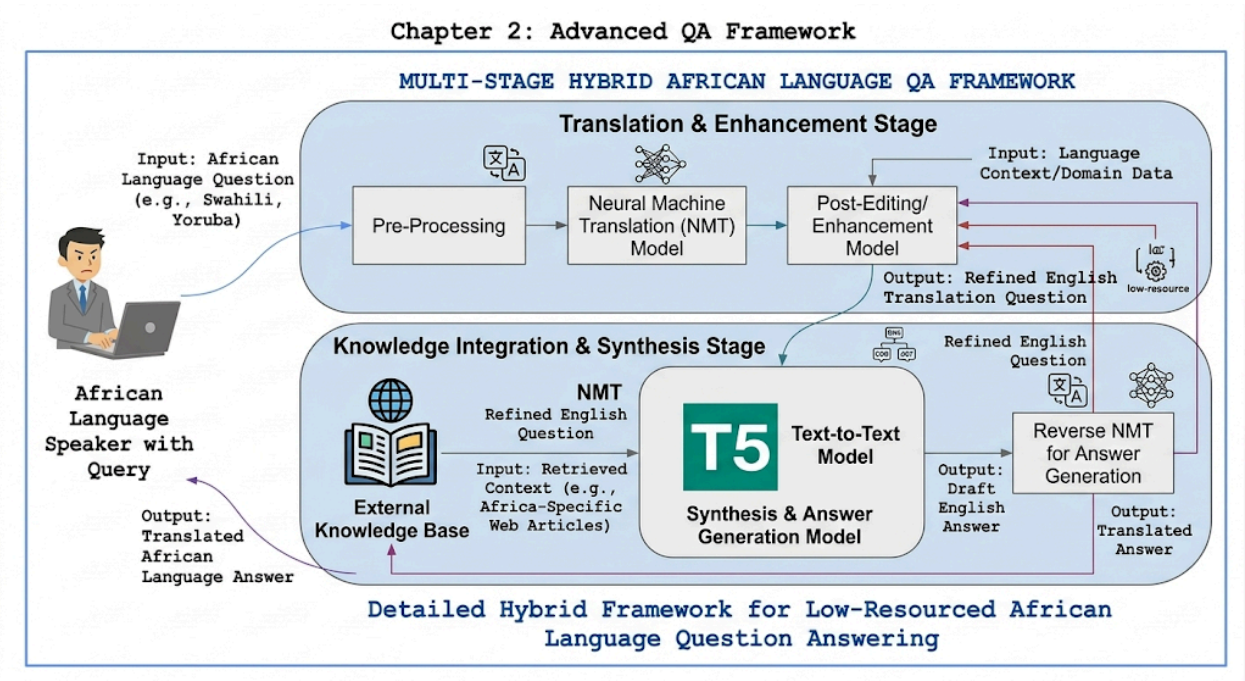
Narrative Flow:

1. A user (e.g., midwife in Kenya) submits a voice query in Swahili: *"Je, mimba ya miezi 6 inahitaji vitamini gani?"* (*"What vitamins are needed at 6 months pregnancy?"*).
2. The system:
    - Detects language/dialect (*Input*).
    - Retrieves WHO guidelines and local clinic protocols via RAG (*Processing*).
    - Generates a response validated by a clinician (*Output*).
3. Feedback from the midwife improves the model's future accuracy (*Iteration*).

Figure 2.1: Conceptual Framework Diagram



Generated using Gemini 3.5 Flash - PRO



Key Considerations

- **Alignment with Objectives**:
    - *Objective IV* (Design): Directly addressed by the AI model and validation modules.
    - *Objective V* (Test): Framework includes feedback loops for iterative improvement.
- **Supervisor Collaboration**: The final diagram will be refined with advisor input to ensure theoretical/practical coherence.

To-Do: End of Lesson 4: - complete by next class

- With reference to your objectives and supervisor's input, complete chapter 2. We still need to see your references growing in the reference section.

Lesson 5: Chapter 3: Methodology

3.1 Introduction

This chapter presents an overview of the methodological approach adopted in the study. It outlines the chapter's structure and briefly introduces key components discussed in subsequent sections. Keep this introduction **concise**, as it serves to frame the chapter rather than provide detailed content.

Note:

You must clearly state the Development Paradigm guiding your project — either

Structured Systems Analysis and Design (SSAD/SSADM) or **Object-Oriented Analysis and Design (OOAD)** — as covered in the section at the end of this session **(end of chapter 3).**

Your choice will shape the overall methodology, since some methods align better with SSAD/SSADM while others suit OOAD. You may consult references such as ChatGPT/Gemini or other academic materials to compare these paradigms.

- You can also use the **experimental approach** in your study. You will need to follow the rest of the document as if to fit your approach. Discuss with your supervisor.

For AI Projects, this section is adapted as follows

3.1 Introduction (For AI and IoT Projects)

- The example given below is for AI, but can also be adapted to IoT
    - After this section, there is an example for non AI/IoT projects - probably shorter than AI/IoT sections.
    - Normal systems such as projects in Blockchain/Pure Software/Algorithms will not use the format below. - check the section after.

This section outlines the chapter while introducing the **experimental design** used in the AI-based study. Unlike traditional software engineering, AI projects emphasize **data-driven experimentation**, **model training**, and **validation**.

The chapter covers stages including **data acquisition**, **processing**, **model training**, and **validation** — ensuring a systematic approach toward developing an AI solution for **maternal healthcare Question Answering (QA)**.

3.2 Research Paradigm

This project adopts an **experimental research paradigm**, where hypotheses are validated through **controlled data analysis** and **model training**. The core steps include: - (you should present this section in prose/paragraphs)

- Defining measurable objectives (e.g., improving accuracy of automated responses in maternal healthcare QA).
- Collecting and preprocessing structured and unstructured healthcare data.
- Training and evaluating machine learning models to verify performance.

Example:

In maternal healthcare QA, the system could be trained to **answer patient FAQs** using data from clinical guidelines. An experiment might test whether adding additional context (such as patient history) improves the accuracy of generated answers.

3.2.1 Data Acquisition

Datasets will be sourced from:

- Public health repositories (e.g., **WHO maternal health datasets**, anonymized hospital records).
- Synthetic data generation tools (e.g., **Synthea**) if real-world data is limited, or manually curated QA datasets.

Example Dataset Contents:

Patient_ID	Age	Blood_Pressure	Previous_Complications	Risk_Level
001	32	120/80	None	Low
002	28	140/90	Preeclampsia	High

Additionally, for QA systems:

Question	Answer - (Gold)
What are symptoms of high blood pressure during pregnancy?	Headaches, vision changes, swelling.
When should I visit the hospital for early labor signs?	If you experience regular contractions, bleeding, or water breaking.

Important:

Ensure compliance with data privacy regulations (e.g., **HIPAA**, **GDPR**) when handling patient data.

3.2.2 Data Processing

Data processing will involve:

- **Cleaning:** Handling missing values (imputation or removal), managing outliers (e.g., capping), correcting inconsistencies.
- **Transformation:** Normalizing numerical data (e.g., Min-Max scaling), encoding categorical variables (e.g., One-Hot Encoding).
- **Augmentation (if required -** when your dataset is either imbalance or really small for training**):** Generating synthetic samples to address class imbalance (e.g., using **SMOTE** for high-risk pregnancy cases).

Example:

The "Previous_Complications" column (text data) may be transformed into **binary features** — e.g., a "Preeclampsia" column with values 1 (present) or 0 (absent).

For QA systems:

Preprocessing may also involve **tokenizing text**, **removing irrelevant terms**, and **indexing questions and answers** for faster retrieval or fine-tuning language models.

3.2.3 Model Training

The chosen model (e.g., **Random Forest**, **Neural Network**, **Transformer-based QA model** such as **BERT**) will be trained using:

- **Stratified k-fold cross-validation** to maintain balanced representation (e.g., for risk categories or question types).
- **Hyperparameter tuning** (e.g., **GridSearchCV**) to optimize performance.
- **Feature importance analysis** (where applicable), to identify key predictors (e.g., blood pressure, age).

Example:

A **Transformer-based QA model** (e.g., **BioBERT**) could be fine-tuned to answer patient questions based on a corpus of maternal healthcare guidelines and validated by comparing its generated answers to expert-reviewed responses.

3.2.4 Model Validation and Testing

The model will be evaluated through:

- **Metrics:** Precision, Recall, F1-score (especially important in imbalanced datasets), and **AUC-ROC** for probabilistic outputs.
- **Testing:** On a held-out test set (typically 20%) not used in training. This should not appear anywhere in training.
- **Explainability:** Using tools like **SHAP** or **LIME** to interpret predictions — critical in healthcare settings.

Example:

If a QA model achieves **90% F1-score** in correctly answering safety-critical questions (e.g., when to seek emergency care), it can be considered reliable for deployment in **maternal healthcare QA** applications. - [*In medical environments, we always need* ***human in the loop*** *to make reliable decisions and not using AI fully to determine the outcome on human lives*]

Example: User (asks questions/assign tasks) -> AI (get results) -> human in the loop (review - approve, reject or modify) -> Show User the results.

Note:

- The Numbering of your document subtitles should now adapt automatically to the numbering format of the template that was given in class.

3.2 System Development Methodology

Note:

You should structure the subtopics in this section based on the **specific content and scope of your project**. It is essential to work closely with your supervisor when developing this section, as your methodology may significantly differ from that of other projects.

In this section, clearly **justify your choice of system development methodology**. Explain how the selected methodology supports the nature of your solution and aligns with your project goals.

You may select from various methodologies depending on the type of system you are developing. For example:

- If using an **iterative and user-focused** approach, you may choose **Agile**, and should further specify which Agile framework (e.g., **Scrum**, **Kanban, incremental**).
- If focusing on **user-centered innovation**, **Design Thinking** may be appropriate.
- For **prototyping and fast delivery**, you might adopt **Rapid Application Development (RAD)**.
- If aiming for **continuous integration and delivery**, **DevOps/MLOps** practices can be integrated.
- If following a **structured and phased** process, consider a **Modified Waterfall** approach.
- For projects focused on **waste reduction and efficiency**, **Lean Development** principles may apply.

Ultimately, ensure that your choice of methodology is **well-reasoned** and that you explain how it will guide the **design, implementation, and evaluation** of your solution.

Example 1: *For a QA System in Maternal Healthcare (AI-based Software Project)*

For this project, the **Agile methodology**, specifically the **Scrum framework**, was selected to manage the iterative development of the Question Answering (QA) system

for maternal healthcare. Agile is well-suited to AI-driven projects where model performance needs continuous refinement through feedback and experimentation.

By using Scrum, the project team can deliver working components in short sprints — for example, progressively improving the QA model's ability to answer questions related to antenatal care, high-risk symptoms, or nutritional advice.

Additionally, the flexibility of Agile allows rapid incorporation of healthcare expert feedback after each sprint, ensuring the system remains accurate and clinically relevant.

Example 2: *For an IoT-based Maternal Healthcare Monitoring System*

This project adopts a **Modified Waterfall methodology** to guide the development of the IoT-based maternal healthcare monitoring system. The system involves both **hardware components** (wearable sensors) and **software services** (data processing, alert generation), which benefit from a structured, phased development approach.

The Modified Waterfall allows for clear documentation of **hardware integration**, **data transmission protocols**, and **cloud-based analytics**, while also supporting limited iterations during testing phases.

This approach ensures that each phase — from sensor calibration, to network setup, to real-time alert validation — is fully tested before deployment in a sensitive healthcare environment.

What informs the choice of a Development Approach/Methodology?

SSAD / SSADM (Structured Systems Analysis and Design Methodology)

SSAD/SSADM is **process-oriented** and **document-driven** — it works well with **linear or phased methodologies** where the system is designed upfront with clear documentation of requirements, data flows, and system structure.

Suited Methodologies:

✅ **Waterfall / Modified Waterfall**

- Matches the structured, sequential nature of SSAD/SSADM.
- Phases: Requirements → Design → Implementation → Testing → Deployment.

✅ **Rapid Application Development (RAD)** (with structured prototyping)

- Works if you're building prototypes based on **well-documented user requirements** and using **structured data models**.

✅ **V-Model (Verification & Validation Model)**

- Very structured — ensures validation at each stage, complements SSAD's rigorous documentation.

Use Case Example:

Building a hospital record system where data flows and processes must be clearly defined and validated before implementation.

OOAD (Object-Oriented Analysis and Design)

OOAD focuses on **modeling systems as objects** (with attributes and behaviors), and supports **iterative**, **flexible**, and **modular** development — best suited to **incremental and adaptive methodologies**.

Suited Methodologies:

✅ **Agile** (Scrum, Kanban, XP)

- Strong synergy: OOAD encourages **modular, incremental builds** — Agile delivers **working software in iterations**.

✅ **DevOps/MLOps**

- OOAD models (classes/objects) align well with **CI/CD pipelines** and **automated testing** in DevOps practices.

✅ **Lean Development**

- OOAD supports building **minimal viable product (MVP)** architectures which can be expanded gradually — ideal for Lean.

✅ **Design Thinking** (paired with OOAD for innovative solutions)

- OOAD naturally supports **prototyping** and **iterative user feedback**, which complements **Design Thinking.**

Use Case Example:

Developing an AI-powered maternal healthcare chatbot/intelligent conversational

agent where components (conversation engine, QA module, database models) are built as **independent objects**, refined iteratively.

Summary Table:

Development Paradigm	Suited Methodologies
SSAD / SSADM	Waterfall, Modified Waterfall, RAD (structured), V-Model
OOAD	Agile (Scrum, Kanban, XP), DevOps, Lean, Design Thinking

---

3.2.1 Justification of the Methodology

In this section, you should provide a **well-reasoned rationale** for selecting the chosen system development methodology. Clearly explain **why the methodology is appropriate** for your specific project — considering factors such as project complexity, flexibility needs, stakeholder involvement, and the nature of the system being developed (e.g., AI-based, IoT-based, or traditional software).

Your justification must be **supported by relevant literature** — cite authoritative sources (books, journal articles, conference papers, or best-practice guidelines). The goal is to demonstrate that your choice is **not arbitrary**, but informed by established knowledge and aligned with best practices in system development.

Example structure for students:

"The Agile Scrum methodology was selected for this project because it supports iterative development and continuous integration of feedback, which is essential when building AI-driven Question Answering systems for maternal healthcare (Beck et al., 2001; Schwaber & Sutherland, 2020). Since model accuracy and relevance require frequent validation by domain experts, the flexibility and collaborative nature of Scrum align well with project needs. Similar approaches have been successfully applied in healthcare AI projects (Smith et al., 2022)."

3.3 Methodology Diagram

In this section, you will present a **diagrammatic representation** of your chosen methodology. The diagram should be **appropriately referenced** — as you do not own the rights to external diagrams, you must properly **cite the source** of the diagram used.

After presenting the diagram, you should provide a **discussion of each stage of the methodology**, clearly explaining how each stage will be **applied to your specific project**.

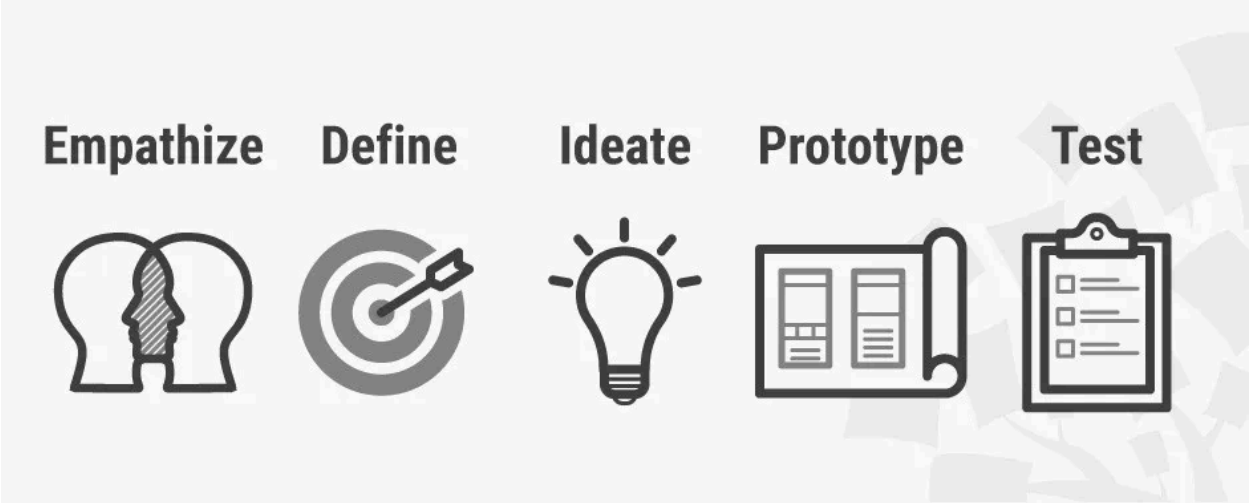
Note: - Follow the Explanation in Class on how this section Changes and has two different formats.

Two common methodology examples include:

- **Design Thinking**

- **Prototyping** (as part of **Agile Development**)

Example 1: **Design Thinking**



3.3.1 Empathize

In this stage, the goal is to **understand the needs, challenges, and context of the users**. It is critical because a deep understanding of user needs ensures that the solution will be both **usable** and **valuable**.

For this project, relevant data will primarily come from **secondary sources** such as published maternal healthcare studies, healthcare guidelines (e.g., WHO, MoH), and existing QA systems in healthcare.

You must explain **how this stage will inform your specific project** — for example, identifying common questions pregnant women ask in QA systems or understanding what types of alerts are most useful in IoT maternal monitoring.

3.3.2 Define

In this stage, you will **clearly define the problem statement** and outline key **model or solution parameters** based on insights from the Empathize stage.

This helps to ensure that the project is solving the **right problem** and that development is focused on clearly defined objectives.

For example, in an AI QA project, you might define the categories of healthcare questions your system will cover (e.g., nutrition, danger signs, antenatal care schedules).

3.3.3 Ideate

This stage involves **brainstorming and exploring potential solutions** to the problem. It encourages **creative thinking** and helps identify the most promising design approaches.

You will also specify the **design paradigm** chosen for your project (e.g., **OOAD** or **SSAD/SSADM**), and explain **why this paradigm was selected**.

The chosen paradigm will influence the type of design diagrams presented in Chapter 4 (refer to the provided table for guidance).

For example, if you choose **OOAD**, your diagrams may include **Class Diagrams**, **Sequence Diagrams**, and **Use Case Diagrams**.

3.3.4 Prototype

In this stage, you will build an **early version** of the solution to test concepts and gather feedback.

Clearly explain your **development approach** — will you use an **iterative approach** (Agile), a phased approach, etc.?

Also describe the **tools and technologies** you will use — for instance:

- Programming languages (e.g., Python, JavaScript)
- Frameworks (e.g., TensorFlow, React, Node.js)
- IoT platforms (e.g., Arduino, Raspberry Pi)
- QA frameworks (e.g., Hugging Face Transformers for NLP-based QA systems).

Justify **why these tools and techniques are appropriate** for your project.

3.3.5 Test

In this stage, you will conduct **rigorous testing** of your solution to ensure it meets the defined requirements and performs reliably.

Explain the **testing approaches** you will use and why they are suitable:

- **Unit Testing:** Testing individual components for correctness.

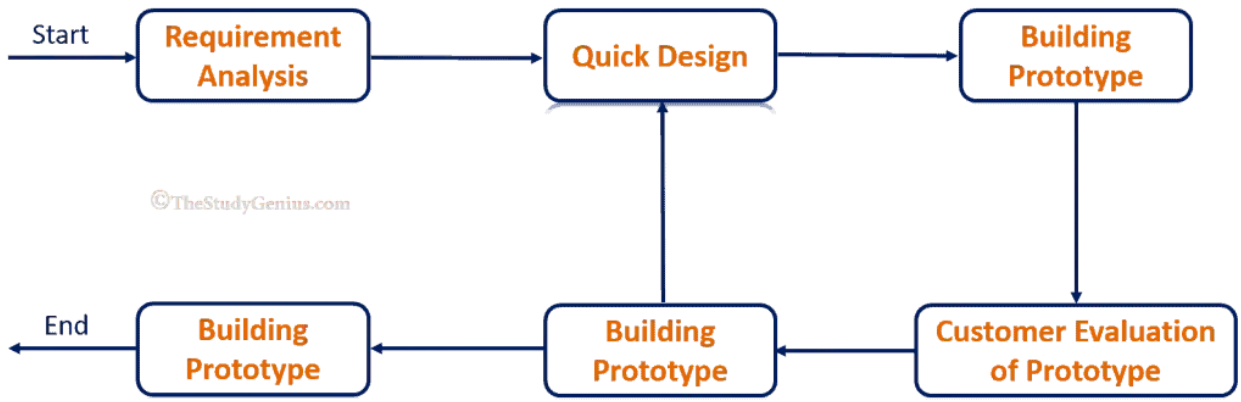
- **Black Box Testing:** Testing the system from a user's perspective without knowledge of internal code.
- **White Box Testing:** Testing internal logic and structure of the code.

You should also explain how you will evaluate **system performance**, for example:

- For QA models: **Accuracy testing, F1-score, precision, recall**.
- For IoT systems: **Sensor accuracy, alert response time, reliability**.

Finally, describe the **test cases** you will design to cover different usage scenarios.

Example 2: Prototyping (as part of Agile Development)



3.3.1 Prototype Planning (Equivalent of "Empathize + Define")

In Prototyping under Agile, the first step is to **understand the user needs** and **define the scope of the prototype**.

You will begin by gathering **user requirements** through secondary sources (published research, guidelines, previous systems) or direct stakeholder input (e.g., interviews with healthcare providers).

Clearly **define what the prototype will demonstrate** — for example:

- For a QA system: answering a core set of maternal health questions.
- For an IoT system: capturing and displaying vital signs from wearable sensors.

You must explain **why these functions are the priority** and how they align with project goals.

3.3.2 Initial Prototype Development (Ideate + Prototype)

This is where you **design and build the first version of the system** (often a minimal viable product or MVP).

You will:

- Describe your **design approach** — iterative and incremental.
- Mention the **design paradigm** (OOAD or SSAD/SSADM), and **why you chose it**.
    - If using OOAD, expect to create **Class Diagrams, Sequence Diagrams, Use Case Diagrams**.
    - If using SSAD/SSADM, expect to use **Data Flow Diagrams, Entity-Relationship Diagrams**.

Then outline your **development tools** and **tech stack**, for example:

- Programming languages
- Frameworks (e.g., Django, Flask, React)
- ML libraries (for QA models)
- IoT platforms (for IoT systems).

3.3.3 Iterative Refinement (Agile Sprints)

Agile-based Prototyping involves **repeated cycles (sprints)** where:

- The prototype is demonstrated to users/stakeholders.
- Feedback is collected.
- The prototype is refined and improved.

Explain how your team will conduct **sprint reviews** or **iteration cycles** — e.g.:

"After each 2-week sprint, the QA system will be tested with a sample of clinical questions. Feedback from domain experts will guide the next sprint's focus."

3.3.4 Testing

Describe your **testing approach** and types of testing used during Prototyping:

- **Unit Testing**: To verify correctness of individual components.
- **Integration Testing**: To ensure system components work together.
- **System Testing**: To test the prototype in an end-to-end scenario.

- **User Acceptance Testing (UAT)**: To gather feedback from end-users or healthcare experts.

Also specify performance testing:

- For QA systems: **accuracy of answers, precision, recall**.
- For IoT systems: **sensor accuracy, latency of data transmission, reliability of alerts**.

Clearly explain **why these testing methods are appropriate** for an **Agile prototyping approach** — which focuses on **iterative improvement** rather than exhaustive testing in the first release.

3.3.5 Final Prototype Evaluation

In this phase, evaluate whether the **prototype meets key project goals** and is ready for:

- Deployment
- Further development
- Scaling to full system

Describe:

- How test results will be documented.
- How stakeholder feedback will inform next steps.

- How success will be measured (e.g. user satisfaction, model accuracy thresholds, system stability).

Points to Note:

In some cases, your supervisor or examiner may require the **Analysis** and **Design** stages to be presented as **separate, dedicated sections** — rather than embedded within the overall methodology flow.

If this is the case, you can structure your Methodology chapter as follows:

- **Section 3.4: Analysis Diagrams**

  Present and explain all diagrams used to analyze system requirements (e.g., Data Flow Diagrams, Use Case Diagrams, Entity-Relationship Diagrams).
- **Section 3.5: Design Diagrams**

  Present and explain diagrams related to system architecture and design (e.g., Class Diagrams, Sequence Diagrams, Component Diagrams).

When using this approach, remember to **cross-reference these sections** within your main methodology stages (Empathize, Define, Ideate, Prototype, Test) — for example:

"Detailed analysis diagrams are presented in Section 3.4."

"The system design is further elaborated in Section 3.5."

Additionally, complete the chapter with:

- **Section 3.6: Deliverables**

  Outline the key outputs or products of the project (e.g., working prototype, QA model, IoT dashboard)

- **Section 3.7: Tools and Techniques**

  List and justify the tools, frameworks, platforms, and techniques used throughout development.

What informs the choice of Diagrams that you should draw for either **OOAD** or **SSDM/SSADM**?

OOAD	SSADM/SSDM	Belongs To
Use Case Diagram	Context Diagram	System Analysis
Use Case Descriptions *(Optional)*	DFD Level 0	System Analysis
Sequence Diagram	DFD Level 1	System Analysis & Design
Activity Diagram	Process Specifications	System Analysis
Class Diagram	ERD	System Analysis & Design
State Machine Diagram *(Optional)*	Structure Chart *(Optional)*	System Design
Database Schema	Database Schema	System Design
UI Mockups / Wireframes	UI Mockups / Wireframes	System Design
System Architecture	System Architecture	System Design
—	Network Topology *(Optional)*	System Design

This marks the end of Chapter 3. The first options of doing Chapter 3 are the best options.

References

All references should be formatted using the **APA referencing style (version/edition 7)**. If you have followed the correct referencing process from the start (see the shared video on how to use mendeley), your reference list should be **automatically generated** by your chosen reference management tool (might be zotero or others).

Important guidelines:

- **Avoid undated references** (e.g., "n.d."), unreliable websites, and non-academic sources such as **Wikipedia**.
- **Prioritize peer-reviewed journal articles**, **conference papers**, **books**, and **official publications** (e.g., WHO reports).

It is highly recommended to use reference management tools such as **Zotero**, **Mendeley**, or similar software to ensure consistent and accurate referencing throughout your document.

Appendices

An **Appendix** contains supplementary material that is not critical to the main text but helps provide **greater depth or additional context** to the research. It may also include material that is too detailed or bulky to fit into the main body of the report.

Examples of suitable content for the appendices:

- **Additional diagrams or images** that complement the Literature Review or Methodology sections (these should be cross-referenced in the main body of the report).
- **Gantt Chart** or **Work Plan** showing the project timeline up to project completion.
    - You may use tools such as **Google Sheets**, **Microsoft Project**, or dedicated Gantt Chart tools — choose tools that help you present a **clear and insightful timeline**.
    - **Note:** The project timeline should run up to the end **of November 2026** (with presentations scheduled before final exams - Reading week).

Formatting:

Use a **clear numbering system** for the appendices:

- **Appendix 1**, **Appendix 2**, and so on.
- Each appendix should be clearly titled.

Example:

- *Appendix 1: Gantt Chart*
- *Appendix 2: Feature Extraction in Convolutional Neural Networks (CNNs)*

Semester II: Documentation & System Development

Recall

Semester I Assessment and Transition

Semester I contributes 35% of the final unit grade: proposal progress (5%), written proposal (15%), and proposal oral defense (15%). Students should confirm that all three marks appear correctly on the AMS, compare them with the results and feedback already communicated, and report only genuine discrepancies through the official channel with supporting evidence.

Before Semester II implementation, students must resolve written, defense, classroom, and supervisor feedback; strengthen Chapters 1–3; and preserve alignment from the problem and gap through the objectives, research questions, literature review, methodology, implementation, testing, and evaluation. A feedback-resolution matrix should record the issue, source, affected section, action, and status. The revised work and implementation plan must be reviewed with the supervisor.

Writing the Final Documentation

Use the past tense for completed activities, the present tense for established knowledge and what the document, figures, or tables show, and the future tense only for work that is genuinely incomplete. Objectives may remain in infinitive form, but the

report must explain how each was achieved. Revise the abstract last so it reflects the actual methods, implementation, results, conclusions, and contribution.

Responsible Use of AI

AI may support brainstorming, literature discovery, data analysis, coding, testing, methodological review, and language improvement. It does not replace the student's judgement, authorship, accountability, or supervisor guidance.

Students must verify AI-assisted facts, citations, code, analyses, and interpretations against original sources or direct testing; protect confidential and personal data; disclose material AI use; maintain an AI-use log; respect copyright, bias, fairness, and reproducibility; and understand and defend every retained output. AI must never be used to fabricate citations, data, participants, experiments, results, metrics, screenshots, findings, or approvals.

To-Do: End of the Recall Session

- Check all Semester I marks on the AMS and report genuine discrepancies through the official channel.
- Resolve feedback using a feedback-resolution matrix and correct Chapters 1–3.
- Update proposal language as activities are completed and revise the abstract last.
- Confirm alignment among the problem, gap, objectives, research questions, methodology, implementation, testing, and evaluation.
- Maintain an AI-use log and verify and disclose material AI assistance.
- Review the revised documentation and Semester II implementation plan with the supervisor.

Chapter 4: System Analysis and Design

4.1 Introduction

Chapter 4 presents the analysis and design of the proposed system. It explains what the system must do, the quality conditions it must satisfy, the data and interfaces it depends on, and the diagrams used to communicate the system's structure and behaviour.

Keep the introduction concise. Briefly outline the sections covered, identify the analysis and design diagrams included, and restate the development paradigm or combination of approaches justified in Chapter 3—for example, Object-Oriented Analysis and Design (OOAD), Structured Systems Analysis and Design (SSAD/SSADM), or a justified data-centric, experimental or hybrid approach. The selected diagrams must fit the project rather than being included merely because they appear in a list.

4.2 System Requirements

System requirements describe the capabilities, qualities, interfaces, data and constraints that the complete system must satisfy. They are not limited to the minimum hardware or software configuration needed to install an application. Requirements should be clear, relevant, feasible, testable and traceable to the problem, objectives, system features and evaluation plan. The requirements-engineering guidance in ISO/IEC/IEEE 29148:2018 applies across different project sizes, products and development methodologies.

You may introduce the section using a statement such as: “The system requirements identified for this project are presented in the following subsections.”

📌 Writing Good Requirements

- Give each requirement a unique identifier, for example FR-01, NFR-01, DR-01 or IR-01.
- Write one clear requirement per statement and avoid vague words such as fast, secure, user-friendly or efficient unless they are supported by measurable criteria.
- Identify the relevant stakeholder, source or rationale where this is useful.
- State how the requirement will be verified through inspection, demonstration, testing, measurement or user acceptance.
- Maintain alignment among requirements, objectives, diagrams, implemented features and test cases.

Where practical, maintain a compact requirements catalogue containing the requirement ID, statement, priority, source or rationale, and verification method. Do not invent thresholds simply to make a requirement appear measurable; justify them using stakeholder needs, the intended operating environment, authoritative guidance, benchmarks or experiments.

4.2.1 Functional Requirements

Functional requirements describe what the system must do under specified conditions. They define services, behaviours, business rules, inputs, outputs and interactions that allow users or external systems to accomplish particular tasks.

Examples of correctly framed functional requirements include:

- FR-01: The system shall allow an authorised user to create an account and authenticate before accessing role-specific functions.
- FR-02: The system shall allow a user to search for and retrieve records using the defined search criteria.
- FR-03: The system shall notify the affected user after a specified transaction has been completed or rejected.

In your own documentation, list and explain only the functions required by your project. If you organise the system into modules—such as authentication, file transfer, data collection, reporting or administration—describe the actor, trigger, expected behaviour, output and important exceptions. The requirement should state what is required; detailed implementation evidence belongs in the implementation chapter.

4.2.2 Non-Functional Requirements

Non-functional requirements describe how well the system must operate and the constraints under which it must work. Select only the quality characteristics relevant to the project. ISO/IEC 25010:2023 provides a current product-quality model that can help students identify suitable quality areas.

Possible areas include:

- Performance efficiency, including response time, throughput and resource use.
- Security and privacy, including authentication, authorisation, confidentiality, integrity and accountability.

- Reliability and availability, including recovery from failures and continuity of service.
- Usability and accessibility for the intended users and operating context.
- Compatibility and interoperability with required platforms or services.
- Maintainability, testability and modifiability.
- Safety, fairness, explainability or human oversight where the project can affect people or other critical decisions.

Use measurable, project-specific statements. Suitable patterns include:

- Under [defined workload and environment], at least [X%] of [specified transactions] shall complete within [Y seconds], as measured using [identified test].
- Only users assigned the required role shall access [specified data or function], as verified through access-control tests.
- The system shall recover from [defined failure] within [specified period] without losing [identified critical data].

For AI/ML projects, model quality, robustness, fairness and explainability may form part of the quality and evaluation requirements. For IoT projects, relevant requirements may include latency, sensor accuracy, connectivity, power consumption and reliability. These examples must still be justified and measured within the actual project context.

4.2.3 Data Requirements

Data requirements apply to all project types, not only AI/ML or IoT. They describe the data the system needs, produces, protects and retains.

Discuss the following where applicable:

- Required inputs, outputs, data sources and ownership.
- Data formats, structures, validation rules and permitted values.
- Data quality, completeness, accuracy, provenance and handling of missing or duplicate records.
- Storage, retention, backup, archival and deletion.
- Privacy, confidentiality, consent and access restrictions.
- Transformations, preprocessing and audit information needed to reproduce the work.

Adapt the discussion to the project. AI/ML projects should address datasets, labels, sampling, train/validation/test separation, representativeness and bias. IoT projects should address sensor types, units, sampling frequency, calibration and missing readings. Information systems should describe records, validation and retention, while cybersecurity projects may focus on logs, telemetry, indicators and provenance.

4.2.4 Interface and Integration Requirements

Interface and integration requirements are also general. They describe how users, external services, software components and hardware devices interact with the system.

Discuss the following where applicable:

- User-interface and accessibility requirements.
- APIs, third-party platforms and external data services.

- Authentication or institutional identity services.
- Hardware devices, sensors, gateways and peripheral equipment.
- Communication protocols, message formats and compatibility constraints.
- Timeouts, retries, validation and error handling when an integration is unavailable or returns invalid data.

For example, an IoT project may describe sensor-to-gateway and gateway-to-cloud communication, while an AI project may describe the interface through which a trained model receives input and returns predictions. A conventional information system may instead focus on web, mobile, payment, messaging or institutional-service integrations.

🔗 Requirements Traceability

Every important requirement should link back to an objective or stakeholder need and forward to a feature, diagram and test. When a requirement changes, review the affected design and evaluation evidence. Discuss significant changes with the supervisor.

Discussion of Diagrams and Their Purposes

System analysis and design diagrams help students visualise, examine and communicate different aspects of a proposed solution. They support:

- Understanding and validating system requirements.
- Defining the system boundary and external interactions.
- Representing data, processes, behaviour and solution structure.
- Identifying omissions, inconsistencies, risks and design alternatives.

- Communicating with supervisors, users, developers and other stakeholders.
- Supporting implementation, testing, evaluation and future maintenance.

Use the diagram-selection table already provided in this guide to identify the diagrams appropriate to your project and chosen paradigm. The table is a guide, not a checklist requiring every diagram. Confirm the final selection with the supervisor.

OOAD projects will normally use relevant UML diagrams. SSAD/SSADM projects will normally use context diagrams and data-flow diagrams. AI/ML, IoT, blockchain, cybersecurity and other specialised projects may require a justified combination of UML, structured-analysis, data-pipeline, block, network, architecture or state-oriented diagrams.

📌 Important Distinction: Context Diagram and DFD Level 0

A context diagram represents the entire system as one process and shows its interactions with external entities. A DFD Level 0 decomposes that system into its major internal processes, data stores and data flows. They are related, but they are not the same diagram.

4.3 System Analysis Diagrams

This section presents the diagrams used to analyse the problem, requirements, actors, processes, data and system boundary. The selected diagrams should be consistent with the paradigm justified in Chapter 3.

For every diagram, use the following presentation pattern:

- Narrative before the diagram: explain what the diagram represents, why it is relevant, its scope and the requirements or objectives it addresses.
- Diagram: use the correct notation and show only the information needed for that view.
- Figure caption: number and caption every diagram according to the course formatting guidance.
- Interpretation after the diagram: explain the important flows, relationships, decisions and implications instead of merely repeating labels visible in the figure.

All labels must remain readable at normal document size. Use consistent names across diagrams and keep the editable source files. A diagram that cannot be read, explained or traced to the project requirements does not provide useful evidence.

4.3.1 Use Case Diagram — Worked Example

A use case diagram provides a high-level view of how external actors interact with the system and which services the system exposes. Identify the system boundary, actors, use cases and associations. Use include, extend and generalisation only when their UML meanings are justified.

The use-case example included in this guide demonstrates the expected narrative–diagram–interpretation sequence. It is an example of a presentation, not a diagram that every student should copy. Each student must develop a diagram based on the actors and functions of the actual project.

Points to Note

- Explain the narrative and relevance of the diagram before presenting it.
- Keep external actors outside the system boundary and use clear verb-based names for use cases.
- Do not add databases, internal code modules or technologies as actors unless they genuinely represent an external interacting system.
- Caption the diagram and interpret its important relationships after the figure.
- Revisit the course formatting tutorial and discuss any uncertain notation with the supervisor.

4.4 System Design Diagrams

System design diagrams describe how the proposed solution will be organised to satisfy the analysed requirements. They may show components, classes, data structures, interfaces, deployment, states, networks, hardware blocks, user interfaces or other implementation-oriented views.

Use the same narrative–diagram–interpretation sequence applied in Section 4.3. Each design diagram should make a clear contribution to the project and remain consistent with the analysis diagrams, requirements and terminology.

4.4.1 Logical Database Schema — Worked Example

For projects that use a conventional database, Chapter 4 should normally present a logical database schema rather than a physical storage design. Show the main entities

or tables, important attributes, primary and foreign keys, relationships and cardinalities. Explain significant normalisation or integrity decisions where relevant.

A physical schema contains implementation-specific details such as exact DBMS data types, indexes, partitions, storage configuration and performance tuning. Include these details only when they are necessary to justify the design or evaluation and have been discussed with the supervisor.

Do not force a relational schema onto every project. A **NoSQL** system may instead show collections or documents and explain embedding and referencing decisions. An AI/ML project without a transactional database may require a data-pipeline or feature-store design, while an IoT project may need a telemetry-storage and data-flow design.

The logical-schema example in this guide is the second worked example. Students should use the same style and academic language for whichever design diagrams are appropriate to their projects.

🛠️ Class Guidance: Diagram Quality and Tools

This is guidance for preparing Chapter 4 and is not a heading that students must copy into the final report. Use tools that support the required notation and allow legitimate, clean export:

- **Visual Paradigm** — suitable for formal UML, DFD, ERD and architecture modelling; its AI diagramming assistant can generate and refine model drafts.
- **StarUML** — suitable for UML, ERD, DFD, C4 and other software-modelling views.

- **PlantUML** — useful for reproducible diagram-as-code workflows, especially use case, sequence, class, activity, component, deployment and state diagrams.
- **draw.io/diagrams.net** — a strong general-purpose choice for context diagrams, DFDs, architecture, block diagrams and network topologies.
- **Microsoft Visio** — suitable for process, UML, ERD and network diagrams where the required licence is available.
- **Figma or Penpot** — better suited to UI wireframes, screen flows and interactive prototypes.
- **Mermaid Chart** — useful for AI-assisted and text-based flowcharts, sequence diagrams, class diagrams, ER diagrams and architecture views.

Tool pricing, trials and AI availability change, so confirm the current terms before relying on a tool. Export using SVG or PDF where supported, or a clear high-resolution PNG. Keep the editable source file. Do not hide, crop or bypass a watermark; use a properly licensed, institutionally available or open tool.

🤖 Responsible AI-Assisted Diagramming

AI can help generate a first draft, convert a narrative into diagram code, suggest missing interactions or improve layout. It does not determine whether the project requirements are correct, and it does not remove the student's responsibility to understand and defend the model.

- Verify every actor, use case, process, relationship, data store, component and label against the approved project requirements.

- Do not accept invented functionality merely because an AI-generated diagram looks professional.
- Record the tool, model name where disclosed, exact prompt, important iterations, retained output, corrections and verification in the AI-use log.
- Do not upload confidential, personal, proprietary or identifiable project data to a public AI tool without authorisation and appropriate safeguards.
- Use AI output as a draft that must be reviewed and corrected, not as evidence that the design is valid.

To-Do: End of the Chapter 4 Session

This activity compares your own modelling decisions with AI-assisted diagram generation. The comparison—not the attractiveness of the first AI output—is the main evidence being assessed.

Common Use-Case Narrative

An online project-supervision portal allows a student to authenticate through the university identity service, upload a project document, submit a revision and request a supervision meeting. A supervisor reviews submissions, provides feedback, approves a submission or requests a revision, and confirms meeting requests. A project coordinator assigns supervisors, monitors submission status and schedules proposal or final defences. When a submission, review decision, meeting confirmation or defence schedule is recorded, the system sends a notification through an external notification service. Users may access only the functions permitted by their assigned roles.

Use This Same Instruction with Each AI Tool

Using only the supplied narrative, produce a UML use case diagram. Show the system boundary, external actors, use cases and associations. Use include, extend or generalisation only where logically justified. Do not invent requirements that are not present in the narrative. Return an editable diagram or diagram source where the tool supports it.

What You Must Do

- Step 1 — Read the narrative and draw a UML use case diagram yourself using an appropriate diagramming tool. Complete the manual diagram before consulting an AI diagram generator.
- Step 2 — Research at least three AI-assisted diagramming tools or model-based workflows. Compare their notation support, editability, privacy, export quality, cost or access, and reproducibility.
- Step 3 — Select two AI-assisted options and give each the exact same narrative and instruction shown above.
- Step 4 — Record the product, model name if disclosed, access date, exact prompt, number of refinement attempts and any manual corrections. If the underlying model is not disclosed, state that instead of guessing.
- Step 5 — Compare your manual diagram with both AI-generated diagrams using the criteria below.
- Step 6 — Correct the strongest diagram manually before using it in Chapter 4. You must understand and be able to defend every element that remains.
- Step 7 — Discuss the final diagram and your comparison with the supervisor.

Comparison Criteria

- Coverage and traceability to the supplied narrative.
- Correct identification of actors, use cases and the system boundary.
- Correct UML notation and use of relationships.
- Absence of invented or unsupported functionality.
- Readability, organisation and consistency.
- Editability and reproducibility of the output.
- Time, effort and number of iterations required.
- Privacy, transparency and responsible-use considerations.

✅ Required Submission

- Your manually produced use case diagram.
- The two AI-generated diagrams.
- A concise comparison matrix using the stated criteria.
- The corrected final diagram and a short justification for the approach selected.
- The relevant AI-use log entries, including prompts and verification performed.

The purpose of this exercise is not to prove that AI is better than manual modelling. It is to determine where AI assists the modelling process, where it introduces errors, and what human judgement is required to produce a clear, accurate and defensible system design.

Chapter 5: System Implementation, Testing and Evaluation

Note: As you read these notes, please refer to the shared sample documentations in your Google Classroom.

5.1 Introduction

Chapter 5 explains how the approved design was implemented, made available in a suitable environment, tested and evaluated. It should show what was actually built, how the components work together, the evidence produced during implementation, and the extent to which the completed solution satisfies the objectives and requirements.

The organisation of this chapter will not be identical for every project. Select and number only the subsections that fit the project type, then agree on the final structure with the supervisor. The routes below cover general software and information systems, AI and machine learning, IoT and embedded systems, cybersecurity, blockchain and other specialised or hybrid projects. They are guides—not headings that every student must copy. Chapter 5 should finish with evaluated results, discussion and limitations; the final conclusions, recommendations and future works belong in Chapter 6.

📌 Writing Perspective and Evidence

Write mainly in the past tense because implementation and testing describe completed work. Use the present tense when explaining what the finished system, table or figure shows. Write in the third person and support claims with traceable evidence such as configuration records, repository history, test results, logs, measurements, figures and

supervisor-verified demonstrations. Screenshots may support the discussion, but screenshots alone are not proof that a requirement was met.

5.2 Implementation Environment

The implementation environment describes where the solution was developed, tested and deployed, together with the hardware, software, services and configuration required to reproduce or use it. Distinguish the development environment from the test, staging, pilot or production environment. Do not state that a system is publicly available unless it has actually been deployed for public access; describe a local prototype, controlled pilot or private deployment accurately.

State the minimum configuration that was actually tested and justified. Avoid copying outdated platform examples or vendor marketing specifications. Versions, service tiers and hardware thresholds should come from the implemented solution, measurements, official documentation or controlled experiments.

5.2.1 Hardware and Deployment Environment

Describe the client devices, servers, cloud resources, storage, network connectivity and specialised hardware used by the project. Explain both the development or training equipment and the minimum environment required to run the finished solution. Where the project uses a managed cloud service, record the relevant service, region where material, instance or tier, storage, network assumptions and scaling limits without exposing credentials.

Use a compact table such as **Table 5.1.** Replace the guidance text with project-specific values and evidence. A stated minimum must be supported by a test, measurement, datasheet or provider configuration rather than guesswork.

Table 5.1: Implementation environment and minimum requirements

Item or layer	Minimum tested specification or version	Purpose and justification	Evidence or source
Client device	State processor, memory, storage, operating system and connectivity.	Explain the intended user tasks and the tested minimum.	Device test, measurement or official documentation.
Server or cloud runtime	State compute, memory, storage, service tier and network.	Explain workload, availability and scaling assumptions.	Deployment configuration and benchmark.
Specialised hardware	State the device, sensor, gateway, accelerator or peripheral.	Explain its role, interface and selection reason.	Datasheet, calibration record or experiment.

Additional Evidence for IoT and Embedded Projects

For an IoT or embedded project, identify each sensor, actuator, microcontroller, gateway, communication module, power component and external service that materially affects the solution. Explain its role, interface, operating range, calibration or accuracy where relevant, and why it was selected. Show the end-to-end chain from sensing or input, through local processing and communication, to the action, alert or stored record.

Use one clear, properly attributed image for each distinct component only where the image helps identification. A part number or model is often more useful than a decorative picture. Table 5.2 provides a suitable structure.

Table 5.2: IoT or embedded component evidence

Component	Image or model/part number	Role and connection	Key specification and justification
Sensor or input device	Insert one clear image or state the model/part number.	State the variable, unit, interface and connection.	State the range, accuracy, calibration and selection reason.
Actuator, controller or gateway	Insert one clear image or state the model/part number.	State the control, processing or communication role.	State the power, protocol or performance specification and selection reason.

5.2.2 Software, Services and Configuration

Identify the operating systems, runtimes, frameworks, libraries, databases, application servers, package managers, container tools, mobile platforms, cloud services and external APIs required to build and run the project. Record the exact versions or version ranges that were tested, their purpose, and the reason they were selected.

Discuss Docker, npm, Composer, Firebase, messaging services, email services or similar technologies only when they were genuinely used. Explain the configuration and role of each relevant dependency rather than listing product names. If containers are used, document the images, environment variables, volumes, ports and deployment

differences; the Docker Compose production guidance is a useful starting point. Never expose passwords, API keys, tokens, private certificates or personal data in the report, screenshots or repository.

5.2.3 Deployment and Reproducibility

Provide a concise, reproducible account of how the solution is installed, configured, started and verified. Include the dependency or environment files, database migrations, seed or sample data where lawful, model or firmware version, build commands, deployment steps, and a simple health check or demonstration procedure. State any assumptions, paid services, licences, restricted datasets or network dependencies that another authorised person would need.

🧭 Select the Route That Fits the Project

- General software and information systems — emphasise application modules, database implementation, interfaces, access control, integrations, deployment and user workflows.
- AI and machine learning — emphasise dataset provenance, data preparation, experiment design, model training and selection, evaluation, model packaging, inference and monitoring.
- IoT and embedded systems — emphasise component assembly, circuit and firmware implementation, protocols, calibration, power, connectivity, gateway or cloud integration and end-to-end operation.

- Cybersecurity — emphasise the authorised test environment, assets and threats, control implementation, detection or protection logic, safe attack simulation, evidence handling and repeatable evaluation.
- Blockchain or distributed systems — emphasise network choice, smart-contract or protocol implementation, identities and keys, transaction flow, deployment, cost, security and performance.
- Other or hybrid projects — define an evidence path that matches the objectives, system boundary and evaluation plan, then confirm it with the supervisor.

5.3 System Implementation

Describe implementation in an order that matches the architecture, modules, data or experimental pipeline shown in Chapter 4. For each major component, link the relevant requirement and design, explain the chosen technology and implemented behaviour, show how it integrates with other components, and identify evidence that it works. Record material deviations from the approved design and justify them. Do not turn the chapter into a code listing; use short extracts only when they explain a non-obvious algorithm, interface, configuration or safety decision.

5.3.1 Implemented Modules, Integration and User Workflows

For a general software or information-system project, organise implementation around the modules and the user's tasks. A suitable sequence is database implementation, application modules, integration and user workflows, followed by testing and discussion. Use headings that describe the actual system and adapt the numbering with your supervisor. The example below uses a maternal healthcare appointment and

information portal; the wording is illustrative and should be replaced with evidence from the student's implementation.

🗃️ Database and Data Implementation

Explain how the Chapter 4 data design was implemented: tables or collections, important fields, relationships, validation rules and access permissions. Identify any imported records or sample data and explain their preparation. Show a small, anonymised sample or relevant database extract, then explain how it supports a requirement.

✍️ Example: "The database stored user accounts, appointment slots and bookings. Each booking linked a user to an available slot and recorded its status. Validation prevented incomplete bookings, while access permissions restricted users to the records permitted by their role." Adapt this paragraph to the actual schema and explain how duplicate or conflicting bookings were handled.

⚙️ Application Modules and Business Rules

Describe each major module's purpose, inputs, processing and outputs, together with the technology used and the related requirement. For the maternal portal, these might include registration, appointment booking, approved information pages and staff administration. Explain how authentication, permissions and booking rules were implemented. Include short code extracts only where they clarify an important decision.

🔗 Integration and User Workflows

Follow one complete task through the interface, application logic and database. In the portal example, a user signs in, selects an available appointment, submits a booking and receives confirmation. Explain how the booking is stored and becomes visible to authorised staff. Where reminders or external services are used, describe the connection and what happens if delivery fails. Use numbered, captioned screenshots to show meaningful steps, including validation or error messages.

🧪 Testing and Results

Use the test-case format in Section 5.4.2 of these notes to compare expected and actual outcomes. For the portal, test successful booking, missing details, an unavailable slot, unauthorised access and cancellation. Add performance and recovery tests where required. For user acceptance or usability evaluation, report who participated, the tasks attempted and the findings. Discuss whether the evidence supports the objectives; a completed booking test does not by itself establish that users found the portal easy to use.

5.3.2 Data and Model Implementation for AI/ML Projects

For an ML project, organise the chapter around the data-to-model workflow. Elvis's Chapter 5 illustrates this clearly: Dataset Description comes before Model Training and Evaluation, followed by Functional Testing and Results and Discussion. You may therefore use "5.3 Dataset Description" and "5.4 Model Training and Evaluation" in your own report, then renumber the remaining relevant sections. The headings in these

class notes show alternative project routes; select and adapt them with your supervisor.

📊 Dataset Description

Introduce each dataset by name and source, explain why it suits the research objective, and describe what one record contains. Report the actual size, language, topic or class distribution, ownership or licence, inclusion and exclusion criteria, and any privacy restrictions. Explain the contribution of each source when combining datasets. Include a small, captioned sample and a summary of the final dataset; discuss what they show.

✍️ Maternal Healthcare Question Answering — illustrative wording: “The dataset comprised maternal healthcare questions paired with reference answers and their supporting source passages. Each record contained a question, reference answer, source identifier and topic label. The topic labels were used to examine coverage across antenatal, childbirth and postnatal questions.” Adapt this description to the actual records, cite the sources, and add the measured counts and languages. This example does not establish that a dataset has been collected or clinically reviewed.

🧹 Data Preparation and Partitioning

Explain how the raw data became usable model input: missing or duplicate records, corrections, exclusions, feature extraction or tokenisation, and the final training, validation and test split. State counts as well as proportions and justify the split. Keep related records together where needed to prevent leakage; fit learned preprocessing on training data only. Use augmentation or encoding only when appropriate to the task.

For the maternal QA example, describe how question–answer pairs were checked against their source passages, how incomplete records were handled, and how questions were formatted for the selected model. Keep duplicate questions and their paraphrases in the same partition. Explain any document-level grouping and whether evaluation tests new questions from known sources or questions from unseen sources. Show one permitted sample before and after preparation, with a short explanation of the change.

⚙️ Model Training and Selection

Introduce the baseline and candidate models, then explain their inputs, outputs and selection rationale. Record the model version, important hyperparameters, random seeds, training environment, stopping rule and saved artefacts. Justify cross-validation when used; the number of folds must suit the data. Select the final configuration using validation evidence and reserve the test set for final evaluation. Retain configuration files and experiment logs so the reported run can be reproduced.

For a fine-tuned maternal QA model, compare the original pretrained model with the fine-tuned version under the same evaluation conditions. Explain how questions and any supplied context become input and reference answers become training targets. Report the actual learning rate, batch size, epochs and token limits. If the project instead uses retrieval-augmented generation, describe passage preparation, indexing, retrieval and prompting; report model training only if it occurred.

🧪 Model Evaluation and Application Testing

Choose evaluation measures for the task. For maternal QA, explain the selected answer-quality measures and inspect examples of correct, incomplete and unsupported answers. Exact Match or token-level F1 may support comparison with reference answers where suitable, but text overlap alone does not establish clinical correctness. Describe any qualified domain review actually conducted, including the rubric and sample evaluated. Report measured results and limitations before drawing conclusions.

Then show how the selected model works in the application: a user submits a question, the system processes it, and an answer is displayed. Test valid, empty, out-of-scope and unsupported questions, response time, and the intended fallback or referral behaviour. Present expected and actual outcomes using the test-case format in Section 5.4.2 of these notes. In the discussion, connect model results and application tests to the research objectives. Use actual outputs; the example provides a writing sequence, not findings to copy.

5.3.3 Hardware, Firmware and Integration for IoT/Embedded Projects

For an IoT or embedded project, follow the physical-to-software sequence: component assembly, firmware and data acquisition, communication and integration, then testing and discussion. You may use “5.3 Hardware and Firmware Implementation” with these as subsections, adapting the remaining numbering to your project. The example below uses a maternal healthcare monitoring prototype. Describe the devices and behaviour actually implemented, and treat the example wording as illustrative.

🔌 Component Assembly and Connections

Build on the component table in Section 5.2 by explaining how the selected sensors, controller, gateway, power supply and any actuators were connected. State the relevant pins, interfaces and power requirements. Include a labelled photograph of the assembled prototype and relate it to the circuit or architecture in Chapter 4. Explain and justify any changes made during implementation.

✍️ Example: “The prototype connected the selected sensor to a microcontroller, which collected readings and forwarded them to a monitoring dashboard through a gateway. Each transmitted record contained a device identifier, timestamp, measured value and unit.” Replace the generic component names with the actual devices and explain the connections. Distinguish simulated inputs, bench measurements and any authorised participant measurements.

⚙️ Firmware and Data Acquisition

Explain how the firmware initialises the device, samples the sensor, checks readings and prepares records for transmission. State the sampling interval, units, calibration procedure and handling of missing or invalid readings. For the monitoring example, describe how a reading is compared with a reference instrument or controlled input and how measurement error is recorded. If alerts use thresholds, justify their source and show the implemented decision rule; a prototype threshold is not evidence of clinical validity.

📡 Communication and System Integration

Trace a reading from the device through the chosen protocol, gateway or server to storage and display. Explain message format, device authentication, buffering and retry behaviour where implemented. In the example, show how the dashboard associates a reading with the correct device, displays its timestamp and indicates stale data. Describe what happens when the sensor disconnects, the network fails or power is restored, including whether queued readings are recovered without duplication.

🧪 Component and End-to-End Testing

Test individual components before the complete workflow. For the monitoring prototype, present calibration error, reading-to-display delay, successful message delivery and recovery after disconnection, with the test conditions and repetitions. Evaluate power use or communication range where relevant to the requirements. Use Section 5.4.2's test-case format for expected and actual behaviour, including alerts and failures. Retain circuit photographs, serial logs and message traces, and explain what each item demonstrates. Discuss the limits of the tested conditions and any remaining faults.

5.3.4 Cybersecurity, Blockchain and Other Specialised Projects

Cybersecurity projects should define the authorised laboratory or target scope, threat assumptions, datasets or traffic, control or detection implementation, tool configuration and safeguards used to prevent harm. Never test a third-party system without explicit authorisation. Blockchain projects should document the network, contracts or protocol components, deployment addresses or identifiers where safe,

account and key-management approach, transaction lifecycle, oracle or off-chain dependencies and relevant cost or performance decisions.

Projects outside the listed categories should build an equivalent implementation narrative around their actual artefacts and evidence. The guiding question is not “Which template can be copied?” but “What must a reader know and verify to reproduce, operate and evaluate this solution?”

5.4 Testing and Evaluation

Testing should demonstrate whether the implemented system satisfies its requirements and whether the project objectives were achieved. Begin with the test strategy, then present test cases, results, interpretation, defects and retesting. The general concepts in ISO/IEC/IEEE 29119-1:2022 can help distinguish test planning, design, execution, reporting and incident management.

5.4.1 Test Strategy and Acceptance Criteria

- Define the purpose, scope, environment, test data, responsibilities and completion criteria before presenting results. Select complementary test levels and techniques because they answer different questions: unit testing checks individual components, integration testing checks interfaces, system testing checks the complete solution, and user acceptance or usability testing checks fitness for intended users where applicable.
- Use black-box, white-box or grey-box techniques according to the available knowledge and test objective; they are perspectives, not substitutes for test

levels. Include positive, negative, boundary and exception cases. Add performance, reliability, compatibility, accessibility, security or recovery tests when the corresponding requirements exist.

- Map every important test to a requirement, feature or objective. Record the tool and version, configuration, repetitions, workload and acceptance threshold. A test verdict must compare the observed result with a predefined expected result; it must not be assigned because the system appeared to work once.
- For web-security testing, the OWASP Web Security Testing Guide provides project-specific test ideas. Use authorised targets, protect evidence and report vulnerabilities responsibly. An automated scanner or an SQL-injection screenshot is not a complete security evaluation; combine suitable review, negative testing and manual verification.

5.4.2 Test Cases, Defects and Retesting

Present repeatable test cases and their observed outcomes. Table 5.3 is a suitable template. Test data must be lawful, non-sensitive where possible, and sufficiently described for another authorised evaluator to repeat the test. Replace the illustrative rows with cases from the actual project.

Table 5.3: Test cases and results

Test ID	Traceability	Scenario, data and conditions	Expected result	Actual result, evidence and verdict
TC-001	FR-01 / Authentication	Valid and invalid credentials; include	Authorised users gain permitted access; invalid	Record the observed result, evidence

		boundary and negative inputs.	input is rejected safely.	reference and Pass/Fail.
TC-002	NFR-01 / Performance	State the workload, device and network conditions, tool and repetitions.	State the measurable threshold defined in the requirement.	Report the measured result, evidence reference and Pass/Fail.

Where many tests exist, summarise them by module or requirement and place detailed logs in an appendix or repository. Record failed tests, defect identifiers, corrective changes and retest outcomes. Do not delete or conceal a failed result merely because a later version passed.

5.4.3 Project-Type-Specific Evaluation

- General software and information systems — report functional coverage, user acceptance or usability evidence, performance, reliability, security and data-integrity results that correspond to the requirements.
- AI and machine learning — compare against a meaningful baseline; report task-appropriate metrics, class-wise or subgroup results where relevant, confusion or error analysis, variation across runs or folds, and final held-out test performance. Explain false positives, false negatives or other consequential errors.
- IoT and embedded systems — report component and circuit tests, calibration or measurement error, communication range or packet delivery, latency, power behaviour, recovery from disconnection and end-to-end scenario results.

- Cybersecurity — report coverage, detection or prevention effectiveness, false positives and false negatives, time to detect or respond, resource overhead and safely reproduced attack scenarios where authorised.
- Blockchain or distributed systems — report smart-contract or protocol correctness, access and key controls, transaction success, cost, latency, throughput, finality and behaviour under defined failure or network conditions.
- Human-centred or decision-support systems — include usability, accessibility, task completion, user acceptance and appropriate human-oversight evidence in addition to technical performance.

5.4.4 Results Presentation and Interpretation

Use tables for precise comparisons and figures for patterns, distributions, trends or error analysis. A figure should show an insight, not merely that a notebook or test tool was opened. Number and caption each table and figure, state the test conditions, and discuss what the result means for the requirement, objective and intended user.

Separate the expected result from the observed result. Report sample size, repetitions and variability where they affect interpretation. Do not generalise beyond the evaluated environment or population, and do not fabricate results, logs, screenshots, users or measurements.

5.5 Version Control and Implementation Evidence

Provide the project repository link on a readable label such as Project repository and ensure the supervisor can access it. The repository should make development history

and reproducibility visible. GitHub repository guidance explains the role of repositories, branches and collaboration, while GitHub flow provides a simple branch-based workflow.

- Include a clear README with the problem, setup, configuration, run and test instructions; meaningful commits linked to implemented work; justified branches and pull requests where used; issues or another traceable task record; dependency and environment files; release tags or versions for important milestones; and an appropriate licence or access statement.
- Use a suitable .gitignore and remove credentials, secret keys, personal data, large private datasets and generated build files. If such material was committed, removing the visible file is not enough; rotate the exposed credential and follow the platform's remediation guidance.
- Repository screenshots may support the report, but they do not replace an accessible repository and its history. Explain how the selected workflow supported versioning, experimentation, review or collaboration rather than merely listing branch names.

5.6 Author Oversight and Use of AI-Assisted Tools

Where AI-assisted tools were used, include a short declaration naming the tool and provider, explaining what it assisted with and how its output was checked, and confirming that the student remained responsible for the work. Keep the declaration factual and specific to the actual project; replace the example details with the actual tool and workflow.

✍️ Example

OpenAI Codex was used to assist with code review and language refinement in selected sections of this report. Its suggestions were checked against the source code, test results and cited sources. The student made the final decisions and takes full responsibility for the submitted work.

5.7 Evaluation Results, Discussion and Limitations

Organise this section around the research objectives, important requirements or evaluation questions—not around the order in which screenshots were collected. For each objective, state the implemented output, the relevant evidence, the acceptance criterion, the observed result and what the result permits the study to conclude. Use exact measurements, sample sizes, repetitions and uncertainty where they matter.

Discuss the results rather than repeating tables. Explain why important outcomes occurred, compare them with the selected baseline, Chapter 2 literature or justified expectations where appropriate, and analyse failures and consequential errors. A general software project may discuss requirement coverage and user outcomes; an AI project may discuss baseline differences, error patterns and generalisation; an IoT project may discuss calibration, communication and field conditions; and a security or blockchain project may discuss coverage, overhead, cost and behaviour under defined threats or failures.

State limitations honestly and explain their likely effect on the findings. Relevant limitations may involve data quality or representativeness, participants, devices,

infrastructure, deployment duration, operating conditions, security coverage, cost, access, external services or evaluation design. Describe any mitigation already applied, but do not convert an unresolved limitation into a stronger claim. The final synthesis, recommendations and future work belong in Chapter 6.

To-Do: End of the Chapter 5 Session

- Agree on the final Chapter 5 structure with the supervisor and retain only the project-type subsections that apply.
- Complete the implementation-environment tables using tested versions, minimum specifications, justifications and traceable evidence.
- Document the implemented modules, data or experimental pipeline and integrations, and trace them to the Chapter 4 requirements and design.
- Prepare the test strategy, acceptance criteria and project-specific test cases, including expected results, actual results, evidence, defects and retests.
- Present and interpret the important results using suitable tables, figures, measurements, error analysis and limitations.
- Add the accessible repository link, README, version history, relevant branches or pull requests, environment files and reproducibility evidence.
- Add a brief declaration of AI-assisted tool use, where applicable.
- Verify that the report and repository contain no private data, credentials, fabricated evidence or unsupported claims.
- Review the completed chapter, working system, repository and evaluation evidence with the supervisor before submission.

Chapter 5 should allow an informed reader to understand what was built, reproduce the relevant environment and evaluation, follow the evidence, and judge whether the solution meets its stated objectives and requirements.

Chapter 6: Conclusions, Recommendations And Future Works

6.1 Introduction

Chapter 6 closes the chain from the original problem and objectives to the implemented solution and the evidence reported in Chapter 5. It should provide a concise answer to what the project established, what should be done on the basis of the findings, and what credible work remains.

Do not introduce new experiments, features, results or citations in this chapter. Every conclusion, recommendation and future-work proposal must be traceable to evidence, a limitation or an unresolved question already discussed. Keep the sections distinct: conclusions state what the study established; recommendations propose justified action; future works identify research or development that remains to be investigated.

6.2 Conclusions

Begin by briefly restating the problem, purpose and completed solution. Then answer the research questions or objectives in the same order used throughout the report. For each objective, state the strongest relevant result and what it means without

reproducing the whole results chapter. End by identifying the project's contribution and the conditions or boundaries within which the conclusion holds.

✅ Match the Conclusion to the Evidence Route

- Software and information systems — conclude using implemented capabilities, requirement coverage, user acceptance or usability, performance, reliability, security and data-integrity evidence.
- AI and machine learning — state the selected model or approach, fair baseline comparison, final held-out or external evaluation, important error or subgroup findings, uncertainty and the limits of generalisation. For high-stakes uses, state the role of human oversight.
- IoT and embedded systems — conclude using sensing or control accuracy, calibration, latency, packet delivery, power and recovery evidence under the tested operating conditions.
- Cybersecurity — conclude within the authorised threat model using detection or prevention effectiveness, false alarms, coverage, response time and resource-overhead evidence.
- Blockchain or distributed systems — conclude using protocol or smart-contract correctness, transaction success, cost, latency, throughput, finality and the tested security and network assumptions.
- Hybrid projects — combine the relevant evidence routes and report the weakest important component as honestly as the strongest; a successful interface does not compensate for an unevaluated model, sensor, security control or protocol.

Avoid broad claims such as “the project was successful” unless the defined acceptance criteria support them. Do not claim novelty, superiority, safety, clinical readiness or real-world scalability from a single demonstration or an unfair comparison.

6.3 Recommendations

Recommendations should follow directly from the results and limitations and should be written for a named audience such as users, the adopting organisation, system maintainers, developers, researchers or policy makers. Each recommendation should state the action, the responsible audience, the evidence-based reason and any condition or priority that affects implementation.

💡 Write Recommendations That Can Be Acted On

- Recommend deployment or adoption only within the environment, population and safeguards that were evaluated.
- Recommend infrastructure changes only when measurements show a bottleneck in compute, memory, storage, network, power or response time.
- Recommend data collection, calibration, security hardening, user training, human review or monitoring when a documented error pattern or limitation justifies it.
- Prioritise recommendations and separate essential safeguards from optional improvements.
- Do not disguise an unimplemented feature, an unsupported model change or a personal preference as a finding-based recommendation.

6.4 Future Works

Future works should extend the research rather than repair missing promised work. Select a small number of credible directions that arise from the limitations, error analysis or unanswered questions. For each direction, explain the rationale, the factor that would change, the data or resources required and how the extension should be evaluated. Prioritise the most valuable directions instead of listing every possible technology.

🚀 Examples of Evidence-Based Directions by Project Type

- Software and information systems — longer deployment studies, broader usability or accessibility evaluation, integration with additional systems, stronger recovery testing or evaluation in a new organisational context.
- AI and machine learning — larger or more representative data, external validation, fairness or subgroup analysis, calibration, explainability, robustness, drift monitoring, alternative approaches under a controlled comparison, or deployment evaluation with meaningful human oversight.
- IoT and embedded systems — additional calibration, wider field conditions, power optimisation, communication alternatives, hardware revision, durability testing or longer reliability studies.
- Cybersecurity — broader authorised threat coverage, adversarial or evasion testing, additional environments, longer monitoring, reduced false positives or independent security validation.

- Blockchain or distributed systems — larger network trials, alternative consensus or scaling designs, contract audits, privacy improvements, governance evaluation or sustained cost and performance measurement.

A proposed direction is not automatically valuable because it uses a newer framework, larger model or more expensive device. Link every future work item to the evidence and explain how success would be measured. Confirm scope and feasibility with the supervisor.

To-Do: End of the Chapter 6 Session

- Write a conclusion that answers the problem, research questions and objectives using only evidence already reported.
- State the project contribution and the conditions, population, environment or threat model within which the findings hold.
- Draft audience-specific, actionable recommendations and link each one to a result or limitation.
- Draft a prioritised set of future works, each with a rationale, required evidence and proposed evaluation method.
- Remove new results, citations, unsupported superiority claims and repeated Chapter 5 detail.
- Check the full alignment chain from the problem and objectives to implementation, evaluation, conclusions, recommendations and future works.
- Review Chapter 6 and the final evidence boundaries with the supervisor before submission.

A strong Chapter 6 is brief but decisive: it tells the reader what the evidence supports, what should happen next, and what remains genuinely open for future investigation.